CLAUDE CODE SOURCE STUDY · VOL. 03

Turn, Context와 Streaming Protocol

사용자 입력이 model-visible context가 되고 tool 결과와 terminal 상태를 갖춘 turn으로 닫히는 알고리즘을 해부한다

> 프로젝트의 구조를 외우는 대신, 코드가 어떤 상태와 불변조건을 지키며 결정적인 동작을 보장하는지 이해하기 위한 스터디 보고서다.

이 권에서 얻게 될 통찰

이 권을 마치면 while loop가 아니라 message identity, state transition, projection, commit point로 한 turn을 설명할 수 있다.

Fixed source
Claude Code v2.1.88 복원 snapshot
2ca5ddabfed5f220812ea11f029eda03b21bc4c1
2026-09-20 · Korean study edition

READING BASELINE

OBSERVED

먼저, 지금 보고 있는 시스템의 상태를 고정한다

입력은 command·attachment·hook을 거쳐 message가 되고, query()는 model/tool phase를 async generator로 반복한다.

구현에서 직접 확인되는 범위

canonical message, provider partial block, API projection, UI/SDK event는 서로 다른 representation이다. provider adapter는 block index와 variant를 누적하고 늦은 usage·stop reason까지 정산한다. compaction은 손실 변환이며 retry는 관찰 commit point를 고려한다.

운영을 말할 때 남겨야 할 경계

streaming latency와 idle watchdog, token budget, prompt cache, retry, stop hook, compaction이 turn 의미에 영향을 준다.

이 보고서가 추측으로 채우지 않는 것

provider 내부 처리, production별 cache 정책 값, 그리고 누락된 context-collapse 구현은 현재 snapshot만으로 확정할 수 없다. 이후의 설명은 이 공백을 유지한 채, 실제 코드가 보여 주는 구조와 보장만을 다룬다.

읽는 동안 반복할 질문

이 문장은 코드로 관찰한 사실인가, 구조에서 끌어낸 해석인가, 아니면 개선을 위한 제안인가?

SYSTEM READING MAP

OBSERVED

파일이 아니라 책임과 보장의 연결로 읽는다

이 지도는 번호를 따라 구성요소를 외우게 하지 않는다. 각 영역이 어떤 의미를 소유하고, 그 결과가 다음 영역의 판단 근거가 되며, 마지막에 어떤 외부 보장으로 닫히는지를 문장으로 연결한다.

진입 경계

Input `processUserInput`

local language 해석

"local language 해석"가 확인되어야 다음의 "Context" 영역이 "ordered compilation"를 자신의 보장으로 이어 받을 수 있다. 두 구현의 호출 관계만 보지 말고 그 사이에서 전달되는 값의 정체성과 완료 조건을 함께 확인한다.

상태 소유

Context `system/user/system parts`

ordered compilation

"ordered compilation"가 확인되어야 다음의 "Projection" 영역이 "provider contract"를 자신의 보장으로 이어 받을 수 있다. 두 구현의 호출 관계만 보지 말고 그 사이에서 전달되는 값의 정체성과 완료 조건을 함께 확인한다.

관찰·정산

Projection `normalizeMessagesForAPI`

provider contract

"provider contract"가 확인되어야 다음의 "Protocol" 영역이 "model↔tools"를 자신의 보장으로 이어 받을 수 있다. 두 구현의 호출 관계만 보지 말고 그 사이에서 전달되는 값의 정체성과 완료 조건을 함께 확인한다.

의미 연결

Protocol `query/queryLoop`

model↔tools

"model↔tools"가 확인되어야 다음의 "Recovery" 영역이 "정보와 시도 identity"를 자신의 보장으로 이어 받을 수 있다. 두 구현의 호출 관계만 보지 말고 그 사이에서 전달되는 값의 정체성과 완료 조건을 함께 확인한다.

기록·복구

Recovery `retry/compact`

정보와 시도 identity

"정보와 시도 identity"가 확인되어야 다음의 "Output" 영역이 "surface projection"를 자신의 보장으로 이어 받을 수 있다. 두 구현의 호출 관계만 보지 말고 그 사이에서 전달되는 값의 정체성과 완료 조건을 함께 확인한다.

관찰·정산

Output `event + terminal`

surface projection

이 책임은 앞선 판단을 외부에서 관찰 가능한 결과로 닫는다. 이 지점의 실패 처리가 모호하면 앞의 모든 단계가 성공했더라도 시스템 전체는 정산되지 않은 상태로 남는다.

선을 따라갈 때에는 구현 이름보다 인접한 두 책임 사이에서 전달되는 identity, state, completion을 확인한다. 그 셋 중 하나가 사라지는 지점이 구조적 결함이 나타날 가능성이 가장 높은 경계다.

01 입력과 context를 컴파일한다

문자열을 곧바로 API에 보내지 않고 local command와 typed attachment를 해석한다.

이 장의 논증은 다음 질문을 차례로 해결한다

알고리즘	책임 연결 사용자 입력 dispatch	알고리즘	상태 소유 System/User/System context 조립
코드	진입 경계 Discriminated union과 runtime schema는 서로 다른 보증을 맡는다	알고리즘	책임 연결 API message normalization
구분	책임 연결 세 가지 message representation	사례	상태 소유 사례: “config.ts 한 줄을 수정해줘”가 context가 되기까지

코드 독해의 시작점

`src/utils/processUserInput/**` → `src/utils/queryContext.ts` → `src/utils/messages.ts`

각 절은 앞 절의 결론을 다음 절의 전제로 사용한다. 따라서 용어 목록을 외우기보다, 현재 구현이 어떤 상태를 만들고 그 상태가 다음 결정의 조건으로 어떻게 사용되는지 연결해서 읽는다.

ALGORITHM A07 · 독자의 질문

사용자 입력 dispatch은 어떤 시스템 문제를 닫는가

같은 입력 상자에서 shell command, slash command, attachment가 있는 prompt, 일반 model query가 들어온다. 화면 문자열이 같아 보여도 실행 권한과 후속 경로는 다르다.

코드가 필요해지는 이유

하나의 입력 표면에서 local effect와 model-bound message를 안전하게 구분한다.

OBSERVED — processUserInput이 입력 variant를 해석해 result messages, shouldQuery, tool/effort override를 반환하고 UI는 그 결정을 소비한다.

INFERENCE — widget은 keystroke와 submit을 소유하지만 입력 의미는 별도 processor가 소유해 headless surface와 공유한다.

이 절에서 답할 결정 질문

prefix와 payload shape를 어떤 순서로 해석해야 local effect와 model-bound message가 섞이지 않는가?

이 설명을 확인할 코드 src/utils/processUserInput/processUserInput.ts:515-588 src/utils/processUserInput/processUserInput.ts:281-330

ALGORITHM A07 · 선행 지식

문법 이름보다 먼저, 이 코드가 기대하는 실행 모델을 만든다

선행 지식은 용어 사전이 아니다. 아래 설명은 작은 실행 모델을 만든 다음, 그 모델이 src/utils/processUserInput/processUserInput.ts의 결정점에서 어떤 오독을 막는지 되돌려 확인한다.

01
interpreter와 tagged result

문자열을 곧바로 실행하지 않고 작은 언어의 구문으로 분류한 뒤, 각 variant가 가능한 결과를 tag로 구분한다. TypeScript tag는 consumer가 branch를 빠뜨리지 않게 하지만 runtime prefix ambiguity까지 없애지는 않는다.

> **실제 코드로 돌아가면.** 515–588행에서 parse 순서와 shouldQuery가 함께 결정되는 지점을 본다.

이론은 정답이 아니라 코드에 던질 질문이다

상태·계산 모델
`interpreter · command pattern`
단계의 순서나 표현을 바꿔도 같은 상태와 불변조건에 도달하는가?

프로토콜·식별
`tagged union`
생산자와 소비자가 동일한 값·작업·종료 의미를 해석한다는 증거가 있는가?

ALGORITHM A07 · 실제 코드

OBSERVED

사용자 입력 dispatch 의 결정점은 어디에 있는가

입력은 slash command, bash mode, image, normal prompt를 구별하는 작은 language parser 다.

입력 표현. prompt string/content blocks, commands, attachments, 그리고 permission/context

보존되는 작업 상태. result messages, shouldQuery, 그리고 allowed tools/effort override

bash·slash·normal prompt의 결정적 dispatch

src/utils/processUserInput/processUserInput.ts:515-588

```
515
516     // Bash commands
517     if (inputString !== null && mode === 'bash') {
518       const { processBashCommand } = await import('./processBashCommand.js')
519       return addImageMetadataMessage(
520         await processBashCommand(
521           inputString,
522           precedingInputBlocks,
523           attachmentMessages,
524           context,
525           setToolJSX,
526         ),
527         imageMetadataTexts,
528       )
… 57 lines omitted; use the source link for the complete range …
586       ),
587       imageMetadataTexts,
588     )
```

이 발췌는 고정 commit의 실제 코드다. 다음 페이지에서는 문장 순서를 되풀이하지 않고, 각 branch가 상태의 의미를 어떻게 좁히는지 실행 전·중·후로 다시 읽는다.

ALGORITHM A07 · 상태 전이

호출 순서가 아니라, 판단 근거가 어떻게 변하는지를 추적한다

prompt string/content blocks, commands, attachments, 그리고 permission/context은 처음부터 신뢰된 실행 상태가 아니다. result messages, shouldQuery, 그리고 allowed tools/effort override가 앞선 관찰을 보존하기 때문에 뒤의 branch가 같은 작업을 계속 판단할 수 있다.

전제 정렬	**진입 조건을 해석한다** submit hook를 적용한다. 이 결과가 다음 판단 "입력 종류를 판별한다."의 전제가 된다.
표현 정규화	**표현과 선택지를 좁힌다** 입력 종류를 판별한다. 이 결과가 다음 판단 "local command이면 해당 handler를 실행한다."의 전제가 된다.
유효성 판정	**표현과 선택지를 좁힌다** local command이면 해당 handler를 실행한다. 이 결과가 다음 판단 "model-bound content와 attachment를 typed message로 만든다."의 전제가 된다.
결정 확정	**표현과 선택지를 좁힌다** model-bound content와 attachment를 typed message로 만든다. 이 결과가 다음 판단 "shouldQuery와 per-turn overrides를 반환한다."의 전제가 된다.
상태 인계	**후속 owner가 읽을 상태를 남긴다** shouldQuery와 per-turn overrides를 반환한다. 이 결과를 다음 owner가 읽을 수 있을 때 이 절차가 닫힌다.

이 전이가 끝난 뒤 기대할 수 있는 조건

local-only output을 provider input과 구분한다. hook가 수정한 최종 input을 이후 단계가 사용한다. command 결과도 transcript/UI identity를 가진다.

ALGORITHM A07 · 설명의 보장 경계

이 구현이 지키려는 조건과, 설명이 멈춰야 하는 조건

여기서 불변조건은 형식적 증명의 결론이 아니다. 어느 전이가 의도한 의미를 보존했는지 읽고, 어떤 반례를 넣어 설명의 범위를 확인할지를 정하는 학습 도구다.

전이 전후에 확인할 조건

- **상태·계산 모델의 질문으로 읽기.** local-only output을 provider input과 구분한다.
- **프로토콜·식별의 질문으로 읽기.** hook가 수정한 최종 input을 이후 단계가 사용한다.
- **상태·계산 모델의 질문으로 읽기.** command 결과도 transcript/UI identity를 가진다.

최종 출력만 같아도 중간 identity나 effect가 달라질 수 있으므로, 각 조건을 보존하는 transition을 함께 관찰해야 한다.

설명의 범위를 드러내는 반례

- prefix ambiguity
- hook output truncation
- display text와 submit payload drift

이 반례가 나타나면 "알고리즘이 틀렸다"로 끝내지 않고 어떤 state owner와 boundary에서 의미가 갈라졌는지 기록한다.

OBSERVED 입력 의미를 REPL widget이 아니라 별도 processor에 둬 headless와 재사용한다.

RECOMMENDATION 입력 variant와 origin을 명시적 union으로 유지하고 parse 결과를 snapshot test한다.

ALGORITHM A07 · effect와 운영

정상 경로 뒤에 남는 effect와 실패 뒤에 남는 상태를 구분한다

입력은 local effect 경로 또는 provider query 경로로 분기되고, hook나 attachment가 payload를 보강할 수 있다.

effect가 시작되고 관찰되는 경계

bash/slash variant는 model 호출 전 local effect를 만들 수 있어 permission과 표시 문자열을 분리해야 한다.

현재 설계가 선택한 trade-off와 비용

입력 의미를 REPL widget이 아니라 별도 processor에 둬 headless와 재사용한다. 이 선택의 계산·운영 비용은 command lookup과 content 크기에 비례. 일반적으로 O(C + N).로 기술되지만, 비용 표기만으로 중복 effect나 부분 상태의 의미가 해결되지는 않는다.

실패·중단 뒤에 실제로 남을 수 있는 상태

prefix 중첩, hook output truncation, display/submit payload drift는 사용자가 승인한 것과 실행된 것을 다르게 만든다. 이때 함께 대입할 반례는 prefix ambiguity, hook output truncation, 그리고 display text와 submit payload drift이다. 이들을 모두 하나의 "실패"로 접으면 retry와 cleanup이 무엇을 이어받아야 하는지 알 수 없다.

관측이 다음 판단에 답하게 하는 법

origin, parsed variant, displayed value, submitted payload digest를 연결하면 입력 변형을 추적할 수 있다. 로그의 존재보다 중요한 것은 이 관측값이 같은 요청·state·effect를 가리키며, 복구 또는 중단 결정을 실제로 바꿀 수 있는가이다.

ALGORITHM A07 · 현재 상태 장부

고정 snapshot에서 확인된 것과 아직 말할 수 없는 것을 함께 남긴다

이 장부는 앞의 내용을 압축한 요약이 아니라, 다음 독자가 현재 시스템을 다시 구성하기 위한 출발 상태다.

OBSERVED	입력 processor가 bash·slash·normal prompt를 결정적으로 분기하고 shouldQuery를 반환한다.
CONDITIONAL	등록 command와 prefix 규칙이 일치할 때 UI와 headless가 같은 의미를 공유한다.
UNKNOWN	외부 hook가 만든 모든 payload 변형과 실제 truncation 빈도는 확인되지 않는다.
NEXT READ	새 입력 variant를 추가할 때 tag consumer와 display/submit projection을 함께 갱신해야 한다.

다른 입력과 변경안에 전이해 볼 질문

1. 새 prefix가 기존 slash command의 접두어라면 어느 branch가 먼저 선택돼야 하는가?
2. 화면에는 원문을 보이되 hook가 submit payload를 바꾸면 어떤 provenance를 남겨야 하는가?

이 설명을 확인할 코드 src/utils/processUserInput/processUserInput.ts:515-588 src/utils/processUserInput/processUserInput.ts:281-330

ALGORITHM A08 · 독자의 질문

System/User/System context 조립은 어떤 시스템 문제를 닫는가

provider 요청 하나에는 도구 사용법, 모델 특성, cwd, memory, policy, 사용자 override가 함께 들어가지만 이 조각들은 변경 주기와 authority가 다르다.

코드가 필요해지는 이유

도구, 모델, cwd, memory, policy에서 온 조각을 provider 요청에 맞는 context로 만든다.

OBSERVED — queryContext 조립 함수가 default system, user context, system context를 구분하고 최종 provider input을 구성한다.

INFERENCE — 각 subsystem이 fragment 내용을 소유하고, 조립기는 순서·포함 조건·wire representation을 소유한다.

이 절에서 답할 결정 질문

prompt를 단일 문자열로 보지 않고 provenance가 다른 fragment 조립으로 읽으면 어떤 실패를 발견할 수 있는가?

이 설명을 확인할 코드 src/utils/queryContext.ts:44-87

ALGORITHM A08 · 선행 지식

문법 이름보다 먼저, 이 코드가 기대하는 실행 모델을 만든다

선행 지식은 용어 사전이 아니다. 아래 설명은 작은 실행 모델을 만든 다음, 그 모델이 src/utils/queryContext.ts의 결정점에서 어떤 오독을 막는지 되돌려 확인한다.

01

compiler lowering과 configuration composition

여러 고수준 정책 조각을 provider가 이해하는 낮은 수준의 message 배열로 내리는 과정이다. merge는 문자열 연결이 아니라 우선순위와 누락 가능성을 가진 lowering이다.

> **실제 코드로 돌아가면.** 44–87행에서 fragment가 어느 message role로 내려가고 custom prompt가 무엇을 대체하는지 확인한다.

이론은 정답이 아니라 코드에 던질 질문이다

프로토콜·식별
`compiler lowering`
생산자와 소비자가 동일한 값·작업·종료 의미를 해석한다는 증거가 있는가?

구성·변형·성능
`configuration composition · prompt caching`
조립 방식이나 최적화를 바꿔도 핵심 계약이 adapter와 build 변형 안에서 보존되는가?

ALGORITHM A08 · 실제 코드

OBSERVED

System/User/System context 조립의 결정점은 어디에 있는가

prompt는 정적 문자열이 아니라 ordered dependency projection이다.

입력 표현. tools, model, working directories, MCP clients, 그리고 custom prompt

보존되는 작업 상태. defaultSystemPrompt, userContext, 그리고 systemContext

prompt part composition

src/utils/queryContext.ts:44-87

```
44  export async function fetchSystemPromptParts({
45    tools,
46    mainLoopModel,
47    additionalWorkingDirectories,
48    mcpClients,
49    customSystemPrompt,
50  }: {
51    tools: Tools
52    mainLoopModel: string
53    additionalWorkingDirectories: string[]
54    mcpClients: MCPServerConnection[]
55    customSystemPrompt: string | undefined
56  }): Promise<{
57    defaultSystemPrompt: string[]
… 27 lines omitted; use the source link for the complete range …
85   * know about (coordinator mode, memory-mechanics prompt). That's acceptable —
86   * the alternative is returning null and failing the side question entirely.
87   */
```

> 이 발췌는 고정 commit의 실제 코드다. 다음 페이지에서는 문장 순서를 되풀이하지 않고, 각 branch가 상태의 의미를 어떻게 좁히는지 실행 전·중·후로 다시 읽는다.

ALGORITHM A08 · 상태 전이

호출 순서가 아니라, 판단 근거가 어떻게 변하는지를 추적한다

tools, model, working directories, MCP clients, 그리고 custom prompt은 처음부터 신뢰된 실행 상태가 아니다. defaultSystemPrompt, userContext, 그리고 systemContext가 앞선 관찰을 보존하기 때문에 뒤의 branch가 같은 작업을 계속 판단할 수 있다.

전제 정렬	**진입 조건을 해석한다** system prompt part를 계산한다. 이 결과가 다음 판단 "사용자 환경 context를 계산한다."의 전제가 된다.
표현 정규화	**표현과 선택지를 좁힌다** 사용자 환경 context를 계산한다. 이 결과가 다음 판단 "tool/MCP capability를 반영한다."의 전제가 된다.
유효성 판정	**표현과 선택지를 좁힌다** tool/MCP capability를 반영한다. 이 결과가 다음 판단 "custom/append prompt precedence를 적용한다."의 전제가 된다.
상태 인계	**후속 owner가 읽을 상태를 남긴다** custom/append prompt precedence를 적용한다. 이 결과를 다음 owner가 읽을 수 있을 때 이 절차가 닫힌다.

이 전이가 끝난 뒤 기대할 수 있는 조건

순서가 바뀌면 instruction scope와 cache prefix가 달라질 수 있다. provider projection에 불필요한 UI metadata를 넣지 않는다.

ALGORITHM A08 · 설명의 보장 경계

이 구현이 지키려는 조건과, 설명이 멈춰야 하는 조건

여기서 불변조건은 형식적 증명의 결론이 아니다. 어느 전이가 의도한 의미를 보존했는지 읽고, 어떤 반례를 넣어 설명의 범위를 확인할지를 정하는 학습 도구다.

전이 전후에 확인할 조건

- **프로토콜·식별의 질문으로 읽기.** 순서가 바뀌면 instruction scope와 cache prefix가 달라질 수 있다.
- **구성·변형·성능의 질문으로 읽기.** provider projection에 불필요한 UI metadata를 넣지 않는다.

최종 출력만 같아도 중간 identity나 effect가 달라질 수 있으므로, 각 조건을 보존하는 transition을 함께 관찰해야 한다.

설명의 범위를 드러내는 반례

- 숨은 global config
- 같은 정보를 여러 fragment가 중복
- custom prompt가 필수 mechanics를 제거

이 반례가 나타나면 "알고리즘이 틀렸다"로 끝내지 않고 어떤 state owner와 boundary에서 의미가 갈라졌는지 기록한다.

OBSERVED 하나의 giant prompt 대신 provenance가 다른 부분을 조합한다.

RECOMMENDATION fragment id·owner·cache stability·required capability를 manifest로 남긴다.

ALGORITHM A08 · effect와 운영

정상 경로 뒤에 남는 effect와 실패 뒤에 남는 상태를 구분한다

turn 시작마다 현재 tools/model/cwd/settings를 읽어 context parts를 만든다.

effect가 시작되고 관찰되는 경계

중복 fragment는 token과 cache key를 바꾸고, 필수 mechanics 누락은 model 행동을 바꾼다.

현재 설계가 선택한 trade-off와 비용

하나의 giant prompt 대신 provenance가 다른 부분을 조합한다. 이 선택의 계산·운영 비용은 조각 수와 동적 discovery 비용에 비례.로 기술되지만, 비용 표기만으로 중복 effect나 부분 상태의 의미가 해결되지는 않는다.

실패·중단 뒤에 실제로 남을 수 있는 상태

숨은 global config나 custom override의 과도한 대체는 source owner를 추적하기 어렵게 한다. 이때 함께 대입할 반례는 숨은 global config, 같은 정보를 여러 fragment가 중복, 그리고 custom prompt가 필수 mechanics를 제거이다. 이들을 모두 하나의 "실패"로 접으면 retry와 cleanup이 무엇을 이어받아야 하는지 알 수 없다.

관측이 다음 판단에 답하게 하는 법

fragment id, owner, digest, cache stability를 기록하면 최종 prompt 변화의 원인을 찾을 수 있다. 로그의 존재보다 중요한 것은 이 관측값이 같은 요청·state·effect를 가리키며, 복구 또는 중단 결정을 실제로 바꿀 수 있는가이다.

ALGORITHM A08 · 현재 상태 장부

고정 snapshot에서 확인된 것과 아직 말할 수 없는 것을 함께 남긴다

이 장부는 앞의 내용을 압축한 요약이 아니라, 다음 독자가 현재 시스템을 다시 구성하기 위한 출발 상태다.

OBSERVED	defaultSystemPrompt, userContext, systemContext가 분리되어 조립된다.
CONDITIONAL	feature와 tool 집합에 따라 포함되는 fragment가 달라진다.
UNKNOWN	provider cache hit와 production token 비용은 snapshot만으로 확인할 수 없다.
NEXT READ	fragment 추가 시 중복 내용, role, cache-stable 여부와 제거 조건을 함께 검토해야 한다.

다른 입력과 변경안에 전이해 볼 질문

1. cwd fragment 하나가 바뀌면 어떤 context part와 cache key만 달라져야 하는가?
2. custom prompt가 default를 대체할 때 반드시 남아야 할 mechanics는 어디에서 보장되는가?

이 설명을 확인할 코드 src/utils/queryContext.ts:44-87

SOURCE READING · 1/2

OBSERVED

Discriminated union과 runtime schema는 서로 다른 보증을 맡는다

TypeScript 타입은 빌드 뒤 지워지지만 Zod의 literal tag와 union은 외부 경계에서도 메시지 변형을 검사한다.

SDK message runtime union

src/entrypoints/sdk/coreSchemas.ts:1854-1882

```
1854  export const SDKMessageSchema = lazySchema(() =>
1855    z.union([
1856      SDKAssistantMessageSchema(),
1857      SDKUserMessageSchema(),
1858      SDKUserMessageReplaySchema(),
1859      SDKResultMessageSchema(),
1860      SDKSystemMessageSchema(),
1861      SDKPartialAssistantMessageSchema(),
1862      SDKCompactBoundaryMessageSchema(),
1863      SDKStatusMessageSchema(),
1864      SDKAPIRetryMessageSchema(),
1865      SDKLocalCommandOutputMessageSchema(),
1866      SDKHookStartedMessageSchema(),
1867      SDKHookProgressMessageSchema(),
1868      SDKHookResponseMessageSchema(),
1869      SDKToolProgressMessageSchema(),
1870      SDKAuthStatusMessageSchema(),
1871      SDKTaskNotificationMessageSchema(),
1872      SDKTaskStartedMessageSchema(),
1873      SDKTaskProgressMessageSchema(),
1874      SDKSessionStateChangedMessageSchema(),
1875      SDKFilesPersistedEventSchema(),
1876      SDKToolUseSummaryMessageSchema(),
1877      SDKRateLimitEventSchema(),
1878      SDKElicitationCompleteMessageSchema(),
1879      SDKPromptSuggestionMessageSchema(),
1880    ]),
1881  )
1882
```

> 이 발췌는 설명을 위해 재작성한 의사코드가 아니라 고정 commit에서 기계적으로 가져온 실제 줄이다. 긴 범위는 가운데만 생략했으며 링크에서 전체 범위를 확인할 수 있다.

SOURCE READING · 2/2

OBSERVED

문법 아래에 숨은 상태와 책임

앞의 코드는 짧은 문법 표현을 사용하지만, 그 표현이 보장하는 범위와 프로젝트가 기대하는 범위는 같지 않다.

줄을 따라 확인할 관찰점

먼저 볼 표현	각 message schema가 type/subtype literal을 가진다.
그 표현이 만든 조건	SDKMessageSchema가 허용된 variant를 명시적으로 합성한다.
그 표현이 만든 조건	생성된 TypeScript type과 runtime schema의 source가 분리되어 있다.

언어와 런타임은 여기까지 보장한다

control-flow narrowing은 compile time 보조이고 외부 JSON을 신뢰하려면 runtime parse가 필요하다. 이 프로젝트는 schema에서 public type을 생성해 두 층의 표현을 맞춘다.

프로젝트가 이 메커니즘에 맡긴 책임

canonical internal message와 SDK projection을 동일한 타입 하나로 뭉개지 않고 경계에서 직렬화 가능한 variant를 정의한다.

경계를 잘못 읽을 때 생기는 위험

새 variant는 schema, generated type, projection, persistence consumer를 함께 갱신해야 한다.

이 설명을 확인할 코드 src/entrypoints/sdk/coreSchemas.ts:1854-1882

ALGORITHM A09 · 독자의 질문

API message normalization 은 어떤 시스템 문제를 닫는가

내부 session message에는 UI progress와 metadata가 있지만 provider는 더 엄격한 tool_use/tool_result 관계와 message role을 요구한다. 내부 표현을 그대로 보내면 protocol이 깨지고, wire 표현을 다시 저장하면 정보가 줄어든다.

코드가 필요해지는 이유

UI-only/system metadata와 불완전한 tool state를 제거·병합해 API protocol을 만족시킨다.

OBSERVED — messages normalization이 내부 Message[]를 읽어 provider용 sequence와 관계 lookup을 만든다.

INFERENCE — session store가 canonical form을 소유하고 normalization은 provider 경계의 anti-corruption projection을 소유한다.

이 절에서 답할 결정 질문

normalization은 어떤 정보를 제거·병합하며, 그 손실이 canonical session으로 역류하지 않게 하는가?

이 설명을 확인할 코드 src/utils/messages.ts:1989-2028

ALGORITHM A09 · 선행 지식

문법 이름보다 먼저, 이 코드가 기대하는 실행 모델을 만든다

선행 지식은 용어 사전이 아니다. 아래 설명은 작은 실행 모델을 만든 다음, 그 모델이 src/utils/messages.ts의 결정점에서 어떤 오독을 막는지 되돌려 확인한다.

01

normal form과 lossy projection

consumer 규칙에 맞춰 여러 내부 variant를 한 표현으로 수렴시키면 비교와 전송은 쉬워지지만 UI-only 정보는 사라진다. 손실된 결과를 원본처럼 저장하면 반복할수록 의미가 줄어든다.

> **실제 코드로 돌아가면.** 1989–2028행의 filter·merge가 원본 배열을 바꾸는지와 orphan 처리 순서를 본다.

이론은 정답이 아니라 코드에 던질 질문이다

상태·계산 모델
`normal form`
단계의 순서나 표현을 바꿔도 같은 상태와 불변조건에 도달하는가?

프로토콜·식별
`lossy projection`
생산자와 소비자가 동일한 값·작업·종료 의미를 해석한다는 증거가 있는가?

구성·변형·성능
`anti-corruption layer`
조립 방식이나 최적화를 바꿔도 핵심 계약이 adapter와 build 변형 안에서 보존되는가?

ALGORITHM A09 · 실제 코드

OBSERVED

API message normalization 의 결정점은 어디에 있는가

내부 history를 provider가 수용하는 role/content pairing으로 낮춘다.

입력 표현. 내부 Message[], tool/result relationships, 그리고 attachment policy

보존되는 작업 상태. normalized sequence, 그리고 lookup/index

normalizeMessagesForAPI entry

src/utils/messages.ts:1989-2028

```
1989  export function normalizeMessagesForAPI(
1990    messages: Message[],
1991    tools: Tools = [],
1992  ): (UserMessage | AssistantMessage)[] {
1993    // Build set of available tool names for filtering unavailable tool references
1994    const availableToolNames = new Set(tools.map(t => t.name))
1995
1996    // First, reorder attachments to bubble up until they hit a tool result or assistant message
1997    // Then strip virtual messages — they're display-only (e.g. REPL inner tool
1998    // calls) and must never reach the API.
1999    const reorderedMessages = reorderAttachmentsForAPI(messages).filter(
2000      m => !((m.type === 'user' || m.type === 'assistant') && m.isVirtual),
2001    )
2002

… 23 lines omitted; use the source link for the complete range …

2026      if (!errorText) {
2027        continue
2028      }
```

이 발췌는 고정 commit의 실제 코드다. 다음 페이지에서는 문장 순서를 되풀이하지 않고, 각 branch가 상태의 의미를 어떻게 좁히는지 실행 전·중·후로 다시 읽는다.

ALGORITHM A09 · 상태 전이

호출 순서가 아니라, 판단 근거가 어떻게 변하는지를 추적한다

내부 Message[], tool/result relationships, 그리고 attachment policy은 처음부터 신뢰된 실행 상태가 아니다. normalized sequence, 그리고 lookup/index가 앞선 관찰을 보존하기 때문에 뒤의 branch가 같은 작업을 계속 판단할 수 있다.

전제 정렬	진입 조건을 해석한다	API에 보낼 수 없는 local message를 거른다. 이 결과가 다음 판단 "연속 user/assistant representation을 병합한다."의 전제가 된다.
표현 정규화	표현과 선택지를 좁힌다	연속 user/assistant representation을 병합한다. 이 결과가 다음 판단 "tool_use와 tool_result identity를 정렬한다."의 전제가 된다.
유효성 판정	표현과 선택지를 좁힌다	tool_use와 tool_result identity를 정렬한다. 이 결과가 다음 판단 "attachment를 provider content block으로 투영한다."의 전제가 된다.
상태 인계	후속 owner가 읽을 상태를 남긴다	attachment를 provider content block으로 투영한다. 이 결과를 다음 owner가 읽을 수 있을 때 이 절차가 닫힌다.

이 전이가 끝난 뒤 기대할 수 있는 조건

각 tool_result는 대응 tool_use 뒤에 온다. provider role alternation을 만족한다. internal-only metadata는 경계를 넘지 않는다.

ALGORITHM A09 · 설명의 보장 경계

이 구현이 지키려는 조건과, 설명이 멈춰야 하는 조건

여기서 불변조건은 형식적 증명의 결론이 아니다. 어느 전이가 의도한 의미를 보존했는지 읽고, 어떤 반례를 넣어 설명의 범위를 확인할지를 정하는 학습 도구다.

전이 전후에 확인할 조건

- **상태·계산 모델의 질문으로 읽기.** 각 tool_result는 대응 tool_use 뒤에 온다.
- **구성·변형·성능의 질문으로 읽기.** provider role alternation을 만족한다.
- **프로토콜·식별의 질문으로 읽기.** internal-only metadata는 경계를 넘지 않는다.

최종 출력만 같아도 중간 identity나 effect가 달라질 수 있으므로, 각 조건을 보존하는 transition을 함께 관찰해야 한다.

설명의 범위를 드러내는 반례

- variant consumer 누락
- orphan tool result
- projection을 다시 canonical로 저장해 손실 누적

이 반례가 나타나면 “알고리즘이 틀렸다”로 끝내지 않고 어떤 state owner와 boundary에서 의미가 갈라졌는지 기록한다.

OBSERVED session canonical form과 provider wire form을 분리해 내부 진화를 보호한다.

RECOMMENDATION normalization 결과에 source UUID trace를 선택적으로 붙여 debug 가능하게 한다.

ALGORITHM A09 · effect와 운영

정상 경로 뒤에 남는 effect와 실패 뒤에 남는 상태를 구분한다

provider 요청 직전에 lookup을 만들고 invalid 또는 UI-only state를 걸러 wire message를 만든다.

effect가 시작되고 관찰되는 경계

projection은 network payload를 바꾸지만 canonical journal을 덮어써서는 안 된다.

현재 설계가 선택한 trade-off와 비용

session canonical form과 provider wire form을 분리해 내부 진화를 보호한다. 이 선택의 계산·운영 비용은 lookup을 사용하면 대체로 O(M + B), M은 message 수, B는 block 수다.로 기술되지만, 비용 표기만으로 중복 effect나 부분 상태의 의미가 해결되지는 않는다.

실패·중단 뒤에 실제로 남을 수 있는 상태

새 message variant 누락, orphan result, 반복 projection 저장은 tool 관계나 metadata를 영구히 잃게 한다. 이때 함께 대입할 반례는 variant consumer 누락, orphan tool result, 그리고 projection을 다시 canonical로 저장해 손실 누적이다. 이들을 모두 하나의 "실패"로 접으면 retry와 cleanup이 무엇을 이어받아야 하는지 알 수 없다.

관측이 다음 판단에 답하게 하는 법

wire message마다 source UUID trace를 선택적으로 남기면 protocol error를 내부 record로 되돌릴 수 있다. 로그의 존재보다 중요한 것은 이 관측값이 같은 요청·state·effect를 가리키며, 복구 또는 중단 결정을 실제로 바꿀 수 있는가이다.

ALGORITHM A09 · 현재 상태 장부

고정 snapshot에서 확인된 것과 아직 말할 수 없는 것을 함께 남긴다

이 장부는 앞의 내용을 압축한 요약이 아니라, 다음 독자가 현재 시스템을 다시 구성하기 위한 출발 상태다.

OBSERVED	고정 source에는 API용 normalization entry와 관계 lookup이 있다.
CONDITIONAL	알려진 message variant와 pairing 규칙에서는 provider-valid sequence를 만든다.
UNKNOWN	feature-gated 모든 variant와 malformed transcript에 대한 완전성은 확인되지 않는다.
NEXT READ	새 variant 추가 시 canonical serializer, lookup, normalization, UI consumer를 함께 대조해야 한다.

다른 입력과 변경안에 전이해 볼 질문

1. progress message를 제거한 뒤 tool_result의 source UUID를 여전히 찾을 수 있는가?
2. normalized sequence를 resume input으로 저장하면 두 번째 normalization에서 무엇이 더 사라지는가?

이 설명을 확인할 코드 src/utils/messages.ts:1989-2028

DISTINCTION TABLE

OBSERVED

세 가지 message representation

이 구분이 없으면 UI, provider, persistence가 서로를 오염시킨다.

표현	목적	보존/제거
Canonical internal	상태와 identity	UUID, origin, tool metadata
Provider payload	API protocol	허용 role/content만
UI/SDK projection	사람·host 관찰	progress와 presentation metadata

이 표를 읽는 관점

projection은 복사가 아니라 consumer contract다.

SELECTIVE EXECUTION TRACE

OBSERVED

사례: “config.ts 한 줄을 수 정해줘”가 context가 되기까지

이 사례는 4권의 실제 edit effect와 5권의 resume로 이어진다.

진입 경계

Input

일반 prompt로 분류한다.

이때 남은 상태와 판단이 “Attachment”에서 사용할 전제가 된다.

책임 연결

Attachment

cwd·memory·IDE selection을 typed source로 결합한다.

이때 남은 상태와 판단이 “System”에서 사용할 전제가 된다.

책임 연결

System

도구와 정책이 system prompt part를 만든다.

이때 남은 상태와 판단이 “History”에서 사용할 전제가 된다.

책임 연결

History

compact boundary 이후 유효 message를 선택한다.

이때 남은 상태와 판단이 “Provider”에서 사용할 전제가 된다.

책임 연결

Provider

API message와 tool schema로 lowering한다.

이 지점에서 앞선 상태 변화가 외부에서 관찰 가능한 결과로 정산된다.

이 흐름 전체에서 보존해야 하는 것

각 단계는 원본 origin과 tool identity를 잃지 않아야 한다.

어디에서 설명이 틀어지는가

문자열로 너무 일찍 평탄화하면 authority와 재구성 근거가 사라진다.

이 설명을 확인할 코드 src/utils/processUserInput/processUserInput.ts:281-330 src/utils/queryContext.ts:44-87

02 지침은 하나의 prompt가 아니라 provenance가 있는 program input이다

CLAUDE.md, rules, output style, skill은 모두 model behavior를 바꾸지만 발견 시점·우선순위·호출 조건·도구 범위가 다르다. “context에 들어간다”로 평탄화하면 어느 파일이 왜 행동을 바꿨는지 설명할 수 없다.

이 장의 논증은 다음 질문을 차례로 해결한다

코드	책임 연결 CLAUDE.md discovery는 directory walk와 source priority를 결합한다	코드	관찰·정산 Output style은 답변 형식만이 아니라 coding instruction 보존 여부를 바꾼다
코드	효과·위임 Skill frontmatter는 discovery metadata와 실행 capability를 함께 선언한다	구분	책임 연결 지침 channel을 같은 문자열로 부르지 않는다

코드 독해의 시작점

`src/utils/claudemd.ts` → `src/outputStyles/loadOutputStylesDir.ts` → `src/skills/loadSkillsDir.ts` → `src/constants/systemPromptSections.ts`

각 절은 앞 절의 결론을 다음 절의 전제로 사용한다. 따라서 용어 목록을 외우기보다, 현재 구현이 어떤 상태를 만들고 그 상태가 다음 결정의 조건으로 어떻게 사용되는지 연결해서 읽는다.

SOURCE READING · 1/2

OBSERVED

CLAUDE.md discovery는 directory walk와 source priority를 결합한다

memory file은 한 파일을 읽는 기능이 아니라 managed·user·project·local source를 root에서 cwd 방향으로 조립하는 규칙이다.

source order, directory proximity, include contract

src/utils/claudemd.ts:1-26

```
1     /**
2      * Files are loaded in the following order:
3      *
4      * 1. Managed memory (eg. /etc/claude-code/CLAUDE.md) - Global instructions for all users
5      * 2. User memory (~/.claude/CLAUDE.md) - Private global instructions for all projects
6      * 3. Project memory (CLAUDE.md, .claude/CLAUDE.md, and .claude/rules/*.md in project roots) - Instructions checked
into the codebase
7      * 4. Local memory (CLAUDE.local.md in project roots) - Private project-specific instructions
8      *
9      * Files are loaded in reverse order of priority, i.e. the latest files are highest priority
10     * with the model paying more attention to them.
11     *
12     * File discovery:
13     * - User memory is loaded from the user's home directory
14     * - Project and Local files are discovered by traversing from the current directory up to root
15     * - Files closer to the current directory have higher priority (loaded later)
16     * - CLAUDE.md, .claude/CLAUDE.md, and all .md files in .claude/rules/ are checked in each directory for Project
memory
17     *
18     * Memory @include directive:
19     * - Memory files can include other files using @ notation
20     * - Syntax: @path, @./relative/path, @~/home/path, or @/absolute/path
21     * - @path (without prefix) is treated as a relative path (same as @./path)
22     * - Works in leaf text nodes only (not inside code blocks or code strings)
23     * - Included files are added as separate entries before the including file
24     * - Circular references are prevented by tracking processed files
25     * - Non-existent files are silently ignored
26     */
```

> 이 발췌는 설명을 위해 재작성한 의사코드가 아니라 고정 commit에서 기계적으로 가져온 실제 줄이다. 긴 범위는 가운데만 생략했으며 링크에서 전체 범위를 확인할 수 있다.

SOURCE READING · 2/2

OBSERVED

문법 아래에 숨은 상태와 책임

앞의 코드는 짧은 문법 표현을 사용하지만, 그 표현이 보장하는 범위와 프로젝트가 기대하는 범위는 같지 않다.

줄을 따라 확인할 관찰점

먼저 볼 표현	managed와 user memory를 먼저 읽고 project/local source를 나중에 더한다.
그 표현이 만든 조건	project memory는 cwd에서 root까지 찾은 directory를 reverse해 root부터 가까운 directory 순으로 처리한다.
그 표현이 만든 조건	worktree가 main repo 아래에 중첩된 경우 checked-in instruction의 중복 load를 피한다.
그 표현이 만든 조건	@include는 별도 entry로 확장되고 processed path가 cycle을 제한한다.

언어와 런타임은 여기까지 보장한다

directory proximity와 load order는 단순 파일 검색이 아니라 precedence 표현이다. model이 뒤 instruction에 더 주의를 기울인다는 contract는 문자열 concat 순서를 행동 의미로 만든다. include graph는 source tree와 다른 dependency graph이며 cycle·external path·setting source가 별도 경계다.

프로젝트가 이 메커니즘에 맡긴 책임

이 코드는 project instruction owner를 현재 cwd와 setting source에 묶는다. worktree 전환은 파일 path뿐 아니라 어떤 instruction set이 발견되는지도 바꿀 수 있다.

경계를 잘못 읽을 때 생기는 위험

같은 내용이 symlink·worktree·include로 중복되거나 exclude와 additional directory가 다른 규칙을 쓰면 token 비용과 precedence가 조용히 바뀐다.

이 설명을 확인할 코드 src/utils/claudemd.ts:1-26 src/utils/claudemd.ts:820-936

SOURCE READING · 1/2

OBSERVED

Output style은 답변 형식만이 아니라 coding instruction 보존 여부를 바꾼다

markdown file의 frontmatter와 본문은 이름·설명·prompt·keep-coding-instructions policy를 가진 별도 instruction channel이다.

project/user style load와 frontmatter projection

src/outputStyles/loadOutputStylesDir.ts:1-79

```
import memoize from 'lodash-es/memoize.js'
import { basename } from 'path'
import type { OutputStyleConfig } from '../constants/outputStyles.js'
import { logForDebugging } from '../utils/debug.js'
import { coerceDescriptionToString } from '../utils/frontmatterParser.js'
import { logError } from '../utils/log.js'
import {
  extractDescriptionFromMarkdown,
  loadMarkdownFilesForSubdir,
} from '../utils/markdownConfigLoader.js'
import { clearPluginOutputStyleCache } from '../utils/plugins/loadPluginOutputStyles.js'

/**
 * Loads markdown files from .claude/output-styles directories throughout the project
 * and from ~/.claude/output-styles directory and converts them to output styles.
 *
 * Each filename becomes a style name, and the file content becomes the style prompt.
 * The frontmatter provides name and description.
 *
 * Structure:
 * - Project .claude/output-styles/*.md -> project styles
 * - User ~/.claude/output-styles/*.md -> user styles (overridden by project styles)
 *
 * @param cwd Current working directory for project directory traversal
 */
… 49 lines omitted; use the source link for the complete range …
                prompt: content.trim(),
                source,
                keepCodingInstructions,
              }
            } catch (error) {
```

이 발췌는 설명을 위해 재작성한 의사코드가 아니라 고정 commit에서 기계적으로 가져온 실제 줄이다. 긴 범위는 가운데만 생략했으며 링크에서 전체 범위를 확인할 수 있다.

SOURCE READING · 2/2

OBSERVED

문법 아래에 숨은 상태와 책임

앞의 코드는 짧은 문법 표현을 사용하지만, 그 표현이 보장하는 범위와 프로젝트가 기대하는 범위는 같지 않다.

줄을 따라 확인할 관찰점

먼저 볼 표현	project와 user의 output-styles directory를 공통 markdown loader가 찾는다.
그 표현이 만든 조건	파일명은 fallback style name이고 frontmatter가 name과 description을 덮을 수 있다.
그 표현이 만든 조건	본문은 trim된 prompt가 된다.
그 표현이 만든 조건	keep-coding-instructions는 boolean/string을 해석해 undefined와 명시 true/false를 구분한다.

언어와 런타임은 여기까지 보장한다

output style을 CSS 같은 presentation으로 오해하면 prompt composition에서 실제 coding instruction이 유지되거나 대체되는 결정을 놓친다. undefined는 단순 false가 아니라 default policy에 결정을 위임하는 세 번째 상태다.

프로젝트가 이 메커니즘에 맡긴 책임

style loader는 후보와 provenance를 만들고, 최종 system prompt 조립기가 어떤 style을 선택해 어떤 기본 instruction과 합성할지 소유한다.

경계를 잘못 읽을 때 생기는 위험

잘못된 frontmatter coercion이나 project/user collision은 이름은 같지만 다른 행동을 만들 수 있다. cache invalidation이 없으면 파일 변경 뒤 stale prompt를 유지한다.

이 설명을 확인할 코드 src/outputStyles/loadOutputStylesDir.ts:1-79

SOURCE READING · 1/2

OBSERVED

Skill frontmatter는 discovery metadata와 실행 capability를 함께 선언한다

skill은 전체 본문을 항상 prompt에 넣지 않고 frontmatter로 발견되며, 호출 시 model·effort·hooks·allowedTools·fork context 같은 실행 조건을 해석한다.

source path, token estimate, canonical file identity

src/skills/loadSkillsDir.ts:50-135

```
50    import {
51      extractDescriptionFromMarkdown,
52      getProjectDirsUpToHome,
53      loadMarkdownFilesForSubdir,
54      type MarkdownFile,
55      parseSlashCommandToolsFromFrontmatter,
56    } from '../utils/markdownConfigLoader.js'
57    import { parseUserSpecifiedModel } from '../utils/model/model.js'
58    import { executeShellCommandsInPrompt } from '../utils/promptShellExecution.js'
59    import type { SettingSource } from '../utils/settings/constants.js'
60    import { isSettingSourceEnabled } from '../utils/settings/constants.js'
61    import { getManagedFilePath } from '../utils/settings/managedPath.js'
62    import { isRestrictedToPluginOnly } from '../utils/settings/pluginOnlyPolicy.js'
63    import { HooksSchema, type HooksSettings } from '../utils/settings/types.js'
64    import { createSignal } from '../utils/signal.js'
65    import { registerMCPSkillBuilders } from './mcpSkillBuilders.js'
66
67    export type LoadedFrom =
68      | 'commands_DEPRECATED'
69      | 'skills'
70      | 'plugin'
71      | 'managed'
72      | 'bundled'
73      | 'mcp'
74

… 56 lines omitted; use the source link for the complete range …

131
132   /**
133    * Parse and validate hooks from frontmatter.
134    * Returns undefined if hooks are not defined or invalid.
135    */
```

이 발췌는 설명을 위해 재작성한 의사코드가 아니라 고정 commit에서 기계적으로 가져온 실제 줄이다. 긴 범위는 가운데만 생략했으며 링크에서 전체 범위를 확인할 수 있다.

SOURCE READING · 2/2

OBSERVED

문법 아래에 숨은 상태와 책임

앞의 코드는 짧은 문법 표현을 사용하지만, 그 표현이 보장하는 범위와 프로젝트가 기대하는 범위는 같지 않다.

줄을 따라 확인할 관찰점

먼저 볼 표현	skill source는 commands_DEPRECATED·skills·plugin·managed·bundled·MCP로 구분된다.
그 표현이 만든 조건	발견 단계 token estimate는 name·description·whenToUse만 사용한다.
그 표현이 만든 조건	realpath identity가 symlink/overlapping directory의 중복을 줄인다.
그 표현이 만든 조건	frontmatter hook는 runtime schema로 parse하고 path pattern은 별도 normalize를 거친다.

언어와 런타임은 여기까지 보장한다

progressive disclosure는 instruction을 요약한다는 뜻이 아니라 discovery용 metadata와 실행 시 load할 본문을 서로 다른 시간에 비용화한다는 뜻이다. allowedTools와 executionContext는 내용 설명을 넘어 실제 실행 surface를 제한하거나 바꾼다.

프로젝트가 이 메커니즘에 맡긴 책임

skill loader가 provenance와 declarative capability를 만들고 SkillTool/query context가 현재 turn에 실제로 주입·실행할 authority를 결정한다.

경계를 잘못 읽을 때 생기는 위험

description이 부정확하면 model discovery가 어긋나고, canonical identity 없이 같은 skill을 두 번 load하면 precedence와 token budget이 흔들린다. frontmatter가 있다고 permission을 자동 획득하는 것은 아니다.

이 설명을 확인할 코드 src/skills/loadSkillsDir.ts:50-135 src/skills/loadSkillsDir.ts:136-239

DISTINCTION TABLE

OBSERVED

지침 channel을 같은 문자열로 부르지 않는다

각 channel은 source·activation·authority가 달라 behavior change를 추적할 때 별도 provenance가 필요하다.

Channel	언제 들어오는가	보존해야 할 provenance와 경계
Managed/User/Project/Local memory	session·cwd context 구성 시 discovery	source type, path, directory proximity, include chain
Conditional rules	path pattern과 현재 작업 대상이 맞을 때	pattern, matched resource, source setting
Output style	선택된 응답 style을 system prompt에 합성할 때	style source, keep-coding-instructions의 tri-state
Skill discovery	name·description·whenToUse를 노출할 때	canonical file identity, loadedFrom, estimated tokens
Skill invocation	본문과 argument를 실제 turn에 확장할 때	caller, allowedTools, hooks, model/effort/context
System prompt mechanics	tool·model·environment section 조립 때	section id, owner, cache stability, required/replaceable 여부

이 표를 읽는 관점

최종 prompt string만 저장하면 어떤 source가 지침을 만들었는지, 어떤 부분이 cacheable한지, 어느 instruction이 capability를 바꿨는지 재구성할 수 없다.

03 모델·토큰·비용은 서로 다른 control signal이다

token이라는 단어 아래에는 API의 정확한 count, 문자 길이 기반 추정, context window pressure, turn continuation budget, provider usage와 금액이 함께 있다. 이 값을 하나의 숫자로 취급하면 종료·compaction·표시·비용 판단이 서로 오염된다.

이 장의 논증은 다음 질문을 차례로 해결한다

코드	책임 연결 Token budget은 quota 차감이 아니라 turn continuation policy다	코드	책임 연결 정확한 token count와 rough estimate는 실패 mode가 다르다
구분	책임 연결 모델 실행을 결정하는 수치를 분리한다	사례	책임 연결 한 provider call의 resource accounting을 재구성한다

코드 독해의 시작점

`src/query/tokenBudget.ts` → `src/services/tokenEstimation.ts` → `src/cost-tracker.ts` → `src/utils/model/**`

각 절은 앞 절의 결론을 다음 절의 전제로 사용한다. 따라서 용어 목록을 외우기보다, 현재 구현이 어떤 상태를 만들고 그 상태가 다음 결정의 조건으로 어떻게 사용되는지 연결해서 읽는다.

SOURCE READING · 1/2

OBSERVED

Token budget은 quota 차감이 아니라 turn continuation policy다

budget tracker는 누적 global turn token과 최근 증가량을 사용해 추가 iteration을 권할지 멈출지를 결정한다.

continuation threshold와 diminishing-return decision

src/query/tokenBudget.ts:1-80

```
1     import { getBudgetContinuationMessage } from '../utils/tokenBudget.js'
2
3     const COMPLETION_THRESHOLD = 0.9
4     const DIMINISHING_THRESHOLD = 500
5
6     export type BudgetTracker = {
7       continuationCount: number
8       lastDeltaTokens: number
9       lastGlobalTurnTokens: number
10      startedAt: number
11    }
12
13    export function createBudgetTracker(): BudgetTracker {
14      return {
15        continuationCount: 0,
16        lastDeltaTokens: 0,
17        lastGlobalTurnTokens: 0,
18        startedAt: Date.now(),
19      }
20    }
21
22    type ContinueDecision = {
23      action: 'continue'
24      nudgeMessage: string
25      continuationCount: number
… 50 lines omitted; use the source link for the complete range …
76      }
77
78      if (isDiminishing || tracker.continuationCount > 0) {
79        return {
80          action: 'stop',
```

이 발췌는 설명을 위해 재작성한 의사코드가 아니라 고정 commit에서 기계적으로 가져온 실제 줄이다. 긴 범위는 가운데만 생략했으며 링크에서 전체 범위를 확인할 수 있다.

SOURCE READING · 2/2

OBSERVED

문법 아래에 숨은 상태와 책임

앞의 코드는 짧은 문법 표현을 사용하지만, 그 표현이 보장하는 범위와 프로젝트가 기대하는 범위는 같지 않다.

줄을 따라 확인할 관찰점

먼저 볼 표현	agent run이거나 budget이 없으면 이 continuation policy를 적용하지 않는다.
그 표현이 만든 조건	90% 미만이고 diminishing return이 아니면 continuation count와 delta를 갱신하고 nudge를 만든다.
그 표현이 만든 조건	세 번 이상 이어진 뒤 연속 delta가 500보다 작으면 diminishing으로 종료한다.
그 표현이 만든 조건	종료 event는 token 비율뿐 아니라 continuation count와 duration을 남긴다.

언어와 런타임은 여기까지 보장한다

이 로직은 model context의 hard limit도, 사용자의 결제 quota도 아니다. 정해진 budget 안에서 한 turn이 더 진행할 실익이 있는지를 근사하는 policy다. threshold는 correctness theorem이 아니라 경험적 stop rule이며 globalTurnTokens의 측정 의미에 의존한다.

프로젝트가 이 메커니즘에 맡긴 책임

query loop가 이 결정을 읽는 위치가 다음 provider call을 시작할지 결정한다. 따라서 같은 함수라도 tool effect 뒤에 호출하면 이미 발생한 effect를 되돌리지 못한다.

경계를 잘못 읽을 때 생기는 위험

token counter가 retry나 cache token을 어떤 방식으로 누적하는지 달라지면 동일 budget의 행동이 달라진다. agentId path에서 stop이 “작업 중단”인지 “이 policy 비적용”인지 consumer가 구분해야 한다.

이 설명을 확인할 코드 src/query/tokenBudget.ts:1-80

SOURCE READING · 1/2

OBSERVED

정확한 token count와 rough estimate는 실패 mode가 다르다

API count는 provider contract를 반영하지만 network·provider별 제약이 있고, rough estimate는 즉시 계산되지만 언어와 file format에 따라 오차가 크다.

provider-aware countTokens와 rough estimation

src/services/tokenEstimation.ts:84-175

```
84              id: toolUse.id,
85              name: toolUse.name,
86              input: toolUse.input,
87            }
88          }
89
90          // Strip tool_reference blocks from tool_result content (user messages)
91          if (block.type === 'tool_result') {
92            const toolResult =
93              block as Anthropic.Beta.Messages.BetaToolResultBlockParam
94            if (Array.isArray(toolResult.content)) {
95              const filteredContent = (toolResult.content as unknown[]).filter(
96                c => !isToolReferenceBlock(c),
97              ) as typeof toolResult.content
98
99              if (filteredContent.length === 0) {
100               return {
101                 ...toolResult,
102                 content: [{ type: 'text' as const, text: '[tool references]' }],
103               }
104             }
105             if (filteredContent.length !== toolResult.content.length) {
106               return {
107                 ...toolResult,
108                 content: filteredContent,
… 62 lines omitted; use the source link for the complete range …
171
172         const response = await anthropic.beta.messages.countTokens({
173           model: normalizeModelStringForAPI(model),
174           messages:
175             // When we pass tools and no messages, we need to pass a dummy message
```

이 발췌는 설명을 위해 재작성한 의사코드가 아니라 고정 commit에서 기계적으로 가져온 실제 줄이다. 긴 범위는 가운데만 생략했으며 링크에서 전체 범위를 확인할 수 있다.

SOURCE READING · 2/2

OBSERVED

문법 아래에 숨은 상태와 책임

앞의 코드는 짧은 문법 표현을 사용하지만, 그 표현이 보장하는 범위와 프로젝트가 기대하는 범위는 같지 않다.

줄을 따라 확인할 관찰점

먼저 볼 표현	Bedrock은 별도 client와 message normalization을 사용한다.
그 표현이 만든 조건	Vertex는 count API에 허용된 beta만 전달한다.
그 표현이 만든 조건	thinking block이 있으면 count request에도 최소 thinking budget을 붙인다.
그 표현이 만든 조건	API가 숫자를 돌려주지 않거나 실패하면 null로 보존한다.
그 표현이 만든 조건	rough estimate는 기본 4 bytes/token이며 JSON 계열은 더 조밀하다고 별도 ratio를 쓴다.

언어와 런타임은 여기까지 보장한다

null을 0으로 바꾸면 "측정 실패"가 "비용 없음"으로 변한다. rough estimate는 빠른 scheduling·discovery hint에는 유용하지만 exact context admission이나 billing 사실을 보증하지 않는다.

프로젝트가 이 메커니즘에 맡긴 책임

호출자는 목적에 따라 exact, estimate, unknown 중 하나를 선택하고 provenance를 유지해야 한다. skill discovery처럼 본문을 아직 load하지 않은 경우에는 frontmatter estimate만으로 전체 실행 비용을 안다고 말할 수 없다.

경계를 잘못 읽을 때 생기는 위험

provider message normalization과 count request normalization이 다르면 실제 request token과 사전 count가 어긋난다. custom model의 context·pricing metadata가 없으면 usage는 있어도 cost는 UNKNOWN이 될 수 있다.

이 설명을 확인할 코드 src/services/tokenEstimation.ts:84-175

DISTINCTION TABLE

OBSERVED

모델 실행을 결정하는 수치를 분리한다

모든 숫자를 "토큰 사용량"으로 합치지 않고 어떤 결정에 사용되는지와 불확실성을 함께 읽는다.

수치	결정에 쓰이는 곳	오해하면 생기는 오류
Exact input count	context admission·표시·일부 compact 판단	provider/normalization 차이를 무시한 false precision
Rough estimate	skill metadata·빠른 사전 판단	estimate를 hard limit처럼 사용
Context window/max output	model별 request envelope	model alias와 provider override 뒤 stale capability
Turn budget/delta	추가 iteration과 diminishing return	billing quota나 context limit로 오해
Provider usage	input/output/cache/web-search 누적	retry attempt와 logical turn을 혼합
USD cost	model별 pricing projection과 사용자 표시	unknown model을 0원으로 확정

이 표를 읽는 관점

cost-tracker는 session과 model별 usage를 저장·복원하지만, 저장된 lastSessionId가 같을 때만 이전 값을 복원한다. 이는 durable accounting의 scope가 session identity에 묶여 있음을 보여 준다.

이 설명을 확인할 코드 src/cost-tracker.ts:55-143

SELECTIVE EXECUTION TRACE

OBSERVED

한 provider call의 resource accounting을 재구성한다

독자는 request 전 추정치와 response 뒤 사용량을 같은 값으로 덮어쓰지 않고 단계별 evidence로 연결한다.

책임 연결

Select

effective model·provider·beta·max output을 확정한다.

이때 남은 상태와 판단이 "Estimate"에서 사용할 전제가 된다.

책임 연결

Estimate

필요하면 exact count API를 호출하고 실패는 null로 유지한다.

이때 남은 상태와 판단이 "Admit"에서 사용할 전제가 된다.

책임 연결

Admit

context window와 compact/budget policy가 request 시작 여부를 결정한다.

이때 남은 상태와 판단이 "Execute"에서 사용할 전제가 된다.

효과·위임

Execute

streaming request와 retry attempt가 실제 provider usage를 만든다.

이때 남은 상태와 판단이 "Settle"에서 사용할 전제가 된다.

책임 연결

Settle

response usage를 model별 input/output/cache/web-search와 cost로 누적한다.

이때 남은 상태와 판단이 "Persist"에서 사용할 전제가 된다.

책임 연결

Persist

session identity와 함께 last usage/cost를 저장해 같은 session resume에서만 복원한다.

이 지점에서 앞선 상태 변화가 외부에서 관찰 가능한 결과로 정산된다.

이 흐름 전체에서 보존해야 하는 것

사전 count, logical turn budget, 실제 response usage, 금액은 서로 교체할 수 없는 네 evidence다.

어디에서 설명이 틀어지는가

custom provider의 pricing이나 retry별 usage가 불명확하면 금액과 attempt accounting은 UNKNOWN으로 남겨야 한다.

이 설명을 확인할 코드 src/services/tokenEstimation.ts:84-175 src/cost-tracker.ts:55-143

04 Turn state machine과 streaming

query()는 결과 함수가 아니라 event stream과 terminal을 가진 protocol executor다.

이 장의 논증은 다음 질문을 차례로 해결한다

메커니즘	책임 연결 yield는 여러 번 반환하는 문법이 아니다	코드	책임 연결 query()의 event와 return은 다른 채널이다
알고리즘	책임 연결 queryLoop 상태 전이	알고리즘	효과·위임 tool-use/tool-result pairing
코드	책임 연결 Streaming은 텍스트가 아니라 typed event transport다	구분	책임 연결 Turn의 네 phase
사례	효과·위임 사례의 첫 model/tool 왕복		

코드 독해의 시작점

`src/query.ts` → `src/services/api/claude.ts` → `src/services/tools/**`

각 절은 앞 절의 결론을 다음 절의 전제로 사용한다. 따라서 용어 목록을 외우기보다, 현재 구현이 어떤 상태를 만들고 그 상태가 다음 결정의 조건으로 어떻게 사용되는지 연결해서 읽는다.

LANGUAGE MECHANISM · M01 · 개념 모델

OBSERVED

yield는 여러 번 반환하는 문법이 아니다

yield는 실행 프레임을 보존한 채 IteratorResult를 내보내는 suspension point다. return은 같은 프레임을 terminal 상태로 만들고, throw는 정상 완료가 아닌 abrupt completion을 소비자에게 전파한다.

> **먼저 바로잡을 오개념**
>
> "함수를 여러 번 나눠 실행한다"는 말은 입문용 직관으로는 유용하지만, 무엇이 보존되고 누가 재개하며 종료 값이 어디로 가는지를 설명하지 못한다. 이 프로젝트를 읽으려면 producer frame, iterator protocol, consumer loop를 서로 다른 층으로 분리해야 한다.

다음 표는 같은 generator에서 일어나지만 의미가 다른 세 가지 completion을 생산자 프레임, iterator protocol, query, UI의 층으로 나누어 비교한다.

	yield value	return terminal	throw error
생산자 실행 프레임	local variable·instruction pointer를 보존하고 suspended가 된다.	completed가 되어 정상 경로로는 다시 실행되지 않는다.	abrupt completion으로 빠져나가며 내부 catch가 없으면 닫힌다.
AsyncIterator 소비 결과	next()가 Promise<{ value: Y, done: false }>로 이행한다.	next()가 Promise<{ value: R, done: true }>로 이행한다.	next() Promise가 reject되어 consumer의 catch 경로로 간다.
query 의미	stream event, message, progress처럼 turn 중간 관찰을 전달한다.	completed, max_turns, aborted_tools 같은 terminal reason으로 protocol을 닫는다.	정상 terminal로 모델링하지 못한 오류와 중단을 전파한다.
for await 소비	loop body가 실행되어 즉시 projection할 수 있다.	loop를 끝내지만 return value 자체는 loop 변수로 전달되지 않는다.	loop 밖 catch/finally와 상위 error boundary로 이동한다.
UI 영향	부분 응답·진행·tool 상태를 누적 렌더링할 수 있다.	turn 후속 처리와 입력 재활성화가 가능한 시점이 된다.	오류 표시·cleanup·재시도 정책이 작동해야 한다.

표의 행을 가로로 읽으면 문법 차이가 보이고, 세로로 읽으면 하나의 completion이 코드에서 UI까지 어떻게 투영되는지 보인다.

LANGUAGE MECHANISM · M01 · WORKED EXAMPLE 1

예제 1 — 프레임이 실제로 보존되는지 next()로 관찰한다

첫 yield 전후에 local 변수와 instruction pointer가 어떻게 남는지 가장 작은 동기 generator로 확인한다.

설명을 위한 최소 실행 예제

```
function* frameDemo() {
  let count = 0
  count += 1; yield { type: 'progress', count }
  count += 10
  return { reason: 'completed', count }
}

const it = frameDemo()        // 아직 본문은 실행되지 않음
it.next()                     // { value: progress(1), done: false }
it.next()                     // { value: completed(11), done: true }
it.next()                     // { value: undefined, done: true }
```

코드를 실행하면 호출, 생산자 실행, 보존 상태가 다음 순서로 대응한다.

소비자 동작	생산자에서 일어나는 일	관찰 가능한 상태
frameDemo()	generator object만 만든다.	suspendedStart; count는 아직 초기화되지 않았다.
첫 next()	본문을 시작해 첫 yield까지 진행한다.	count=1과 다음 instruction 위치가 프레임에 남는다.
둘째 next()	yield 다음 문장에서 재개한다.	새 호출이 아니라 보존된 count를 11로 만든 뒤 completed가 된다.
셋째 next()	완료된 iterator를 다시 조회한다.	프레임을 재실행하지 않고 done:true를 유지한다.

이 예제에서 가져갈 결론

여러 값을 돌려준다는 표면적 결과보다 중요한 사실은 첫 next()와 둘째 next()가 같은 실행 프레임을 이어 쓴다는 점이다. yield는 그 프레임의 소유권을 잠시 consumer에게 넘기는 protocol 경계다.

LANGUAGE MECHANISM · M01 · WORKED EXAMPLE 2

예제 2 — async generator에서는 next() 자체가 비동기다

await와 yield를 같은 것으로 보지 않고, 내부 비동기 대기와 외부 iterator handoff를 분리한다.

설명을 위한 최소 실행 예제

```
type Event = { type: 'progress'; step: number }
type Terminal = { reason: 'completed'; total: number }

async function* turn(): AsyncGenerator<Event, Terminal, void> {
  let total = 0
  total += await Promise.resolve(2) // 생산자 내부가 기다린다
  yield { type: 'progress', step: total } // consumer가 관찰한다
  total += await Promise.resolve(3)
  return { reason: 'completed', total }
}
```

코드를 실행하면 호출, 생산자 실행, 보존 상태가 다음 순서로 대응한다.

소비자 동작	생산자에서 일어나는 일	관찰 가능한 상태
await it.next()	첫 await와 yield까지 실행한다.	Promise가 {event, done:false}로 이행한다.
생산자 suspended	consumer가 event를 처리하는 동안 total=2가 유지된다.	다음 next()가 오기 전에는 yield 뒤로 진행하지 않는다.
await it.next()	두 번째 await를 거쳐 return에 도달한다.	Promise가 {terminal, done:true}로 이행한다.

이 예제에서 가져갈 결론

await는 생산자 내부의 비동기 의존성을 기다리고, yield는 준비된 관찰을 iterator 경계 밖으로 넘긴다. 둘이 함께 있기 때문에 네트워크·도구 대기와 점진적 UI 갱신을 한 상태 기계 안에서 표현할 수 있다.

LANGUAGE MECHANISM · M01 · WORKED EXAMPLE 3

예제 3 — 수동 소비와 for await는 terminal 관찰 능력이 다르다

AsyncGenerator<Y, R>의 R이 for-await loop body로 전달된다고 오해하지 않도록 두 소비 방식을 비교한다.

설명을 위한 최소 실행 예제

```
// terminal까지 직접 관찰하는 소비자
const iterator = turn()
while (true) {
  const result = await iterator.next()
  if (result.done) { console.log(result.value); break }
  render(result.value)
}

// 간결하지만 terminal value를 버리는 소비자
for await (const event of turn()) {
  render(event)
}
```

코드를 실행하면 호출, 생산자 실행, 보존 상태가 다음 순서로 대응한다.

소비자 동작	생산자에서 일어나는 일	관찰 가능한 상태
manual next()	done discriminant를 직접 검사한다.	done:true branch에서 Terminal 타입으로 좁혀진다.
for await	done:false 값에 대해서만 body를 실행한다.	done:true는 반복 종료 신호이며 value는 body에 들어오지 않는다.
throw/reject	두 방식 모두 예외 경로로 전환한다.	terminal reason과 error를 같은 값으로 평탄화하지 않는다.

이 예제에서 가져갈 결론

프로젝트에서 return Terminal은 protocol 설계상 의미가 있지만, REPL처럼 for await만 사용하는 표면은 terminal payload를 직접 읽지 않는다. 그 표면은 반복 종료와 별도 상태를 이용해 turn 마감을 처리한다.

LANGUAGE MECHANISM · M01 · 실제 프로젝트

OBSERVED

query()의 yield*는 event 전달과 terminal 회수를 분리한다

query()는 queryLoop()의 중간 값을 yield*로 그대로 전달하면서, 내부 generator가 return한 Terminal만 자신의 local 변수로 회수한다. 이 한 줄은 event forwarding과 terminal capture를 동시에 수행한다.

query의 AsyncGenerator 계약과 yield* delegation — src/query.ts:219-239

```ts
export async function* query(
  params: QueryParams,
): AsyncGenerator<
  | StreamEvent
  | RequestStartEvent
  | Message
  | TombstoneMessage
  | ToolUseSummaryMessage,
  Terminal
> {
  const consumedCommandUuids: string[] = []
  const terminal = yield* queryLoop(params, consumedCommandUuids)
  // Only reached if queryLoop returned normally. Skipped on throw (error
  // propagates through yield*) and on .return() (Return completion closes
  // both generators). This gives the same asymmetric started-without-completed
  // signal as print.ts's drainCommandQueue when the turn fails.
  for (const uuid of consumedCommandUuids) {
    notifyCommandLifecycle(uuid, 'completed')
  }
  return terminal
}
```

consumer next()

REPL 또는 QueryEngine이 query iterator에 다음 값을 요청한다.

이 completion은 다음 "query의 yield*"의 입력 의미를 결정한다.

queryLoop yield

done:false event가 query를 거쳐 consumer에게 그대로 전달된다. query frame과 queryLoop frame은 모두 보존된다.

이 completion은 다음 "queryLoop return"의 입력 의미를 결정한다.

wrapper 후처리

정상 return일 때만 command lifecycle completed 알림을 보낸다. throw나 외부 return()이면 주석대로 이 구간을 건너뛴다.

이 completion은 다음 "query return"의 입력 의미를 결정한다.

query의 yield*

요청을 queryLoop에 위임하고 내부 IteratorResult를 기다린다.

이 completion은 다음 "queryLoop yield"의 입력 의미를 결정한다.

queryLoop return

done:true Terminal은 consumer event가 아니라 yield* expression의 결과가 되어 query의 terminal 변수에 들어간다.

이 completion은 다음 "wrapper 후처리"의 입력 의미를 결정한다.

query return

query 자체도 같은 Terminal로 completed가 된다. 수동 next consumer만 payload를 직접 회수한다.

여기서 wrapper의 terminal 의미가 닫히며, 중간 event와 같은 채널로 소비되지 않는다.

이 설명을 확인할 코드 src/query.ts:219-239

LANGUAGE MECHANISM · M01 · 소비자와 UI

OBSERVED

for await가 받는 것은 terminal 이 아니라 done:false event다

REPL은 query()를 한 번 호출하고 iterator가 내보내는 각 event를 즉시 onQueryEvent로 보낸다. 반복이 끝난 뒤의 코드는 turn이 닫혔다는 사실을 알지만, for-await body는 return Terminal 자체를 받지 않는다.

REPL의 for-await consumer — src/screens/REPL.tsx:2788-2804

```
2788      toolUseContext.renderedSystemPrompt = systemPrompt;
2789      queryCheckpoint('query_query_start');
2790      resetTurnHookDuration();
2791      resetTurnToolDuration();
2792      resetTurnClassifierDuration();
2793      for await (const event of query({
2794        messages: messagesIncludingNewMessages,
2795        systemPrompt,
2796        userContext,
2797        systemContext,
2798        canUseTool,
2799        toolUseContext,
2800        querySource: getQuerySourceForREPL()
2801      })) {
2802        onQueryEvent(event);
2803      }
2804      if (feature('BUDDY')) {
```

책임 연결 — queryLoop producer

StreamEvent·Message·progress를 yield하고, 최종 reason은 return한다.

이 층의 관찰은 "query wrapper"가 해석할 수 있는 형태로만 넘어가며, 생산자 frame 전체가 전달되는 것은 아니다.

책임 연결 — query wrapper

yield*로 event를 중계하고 정상 종료 후 command lifecycle을 닫는다.

이 층의 관찰은 "REPL consumer"가 해석할 수 있는 형태로만 넘어가며, 생산자 frame 전체가 전달되는 것은 아니다.

진입 경계 — REPL consumer

for await의 각 event를 onQueryEvent로 projection한다. terminal 값은 body에 들어오지 않는다.

이 층의 관찰은 "QueryEngine consumer"가 해석할 수 있는 형태로만 넘어가며, 생산자 frame 전체가 전달되는 것은 아니다.

책임 연결 — QueryEngine consumer

각 message를 transcript·SDK event·usage state로 projection한다. 이 경로도 for await를 사용한다.

이 층의 관찰은 "UI/host settlement"가 해석할 수 있는 형태로만 넘어가며, 생산자 frame 전체가 전달되는 것은 아니다.

관찰·정산 — UI/host settlement

반복이 끝났다는 사실, 별도 누적 state, catch/finally를 사용해 loading과 오류를 정리한다.

최종 표면은 반복 종료·누적 상태·cleanup을 함께 읽어 turn을 정산한다.

이 구분 때문에 streaming projection, durable transcript, external effect commit, terminal settlement를 하나의 "응답 반환"으로 합치면 안 된다.

이 설명을 확인할 코드 src/screens/REPL.tsx:2788-2804 src/QueryEngine.ts:675-686

LANGUAGE MECHANISM · M01 · 설명의 경계

직관은 출발점이고, 보장은 protocol에서 확인한다

영상의 직관

영상의 "함수를 여러 번 나눠 실행한다"와 "이벤트를 UI로 흘려보낸다"는 설명은 generator를 처음 보는 독자에게 유용한 출발점이다.

현재 snapshot에서 필요한 보정

하지만 현재 snapshot에서는 queryLoop가 단순히 event를 흘리는 데서 끝나지 않는다. State object, yield* terminal delegation, command lifecycle, abort signal, tool-result pairing과 결합된다. 따라서 정확한 결론은 "한 함수를 여러 번 호출한다"가 아니라 "하나의 resumable execution frame을 AsyncIterator protocol로 소비한다"이다.

이 메커니즘이 자동으로 보장하지 않는 것

보장 경계	yield 지점의 backpressure는 iterator 경계에서만 성립한다. upstream SSE client나 내부 queue가 이미 buffering한 데이터까지 자동으로 제한한다고 단정할 수 없다.
보장 경계	throw를 Terminal의 한 variant처럼 취급하면 retry 가능한 model error, 사용자 abort, programmer error의 cleanup 경로가 섞인다.
보장 경계	consumer가 break하면 iterator return()을 통해 generator가 닫히지만, 이미 시작된 fetch·tool·child effect가 저절로 취소된다는 뜻은 아니다. AbortSignal 전달과 finally/settlement가 별도로 필요하다.
보장 경계	yield된 Message가 곧 durable commit이라는 뜻도 아니다. UI projection, transcript write, external effect의 commit point는 서로 다른 시점일 수 있다.

다른 시스템에도 적용할 수 있는 이론

Resumable execution	instruction pointer와 lexical environment를 보존했다가 next/throw/return 요청으로 재개하는 실행 모델이다.
IteratorResult discriminant	done:false일 때 yield type, done:true일 때 return type으로 좁혀지는 tagged union이다.
Delegation	yield*는 내부 iterator의 중간 결과를 전달하고 내부 return value를 expression result로 회수한다.
Pull boundary	consumer의 next()가 다음 진행을 요구한다. 단, 전체 네트워크 pipeline의 buffer 정책까지 자동으로 규정하지는 않는다.
Abrupt completion	throw, return(), break에 따른 iterator close는 정상 terminal reason과 구별해야 한다.

언어·학습 참고자료 ECMAScript GeneratorYield · MDN yield · TypeScript stricter generators · 영상 연계 글: Claude Code async generator

이 설명을 확인할 코드 src/QueryEngine.ts:675-686 src/query.ts:1260-1266 src/query.ts:1508-1522

ALGORITHM A10 · 독자의 질문

queryLoop 상태 전이은 어떤 시스템 문제를 닫는가

한 user turn 안에서 model 요청, tool 실행, retry, compaction, stop hook가 반복된다. 함수의 while loop만 따라가면 같은 continue가 서로 다른 상태 의미를 갖는다는 사실을 놓치기 쉽다.

코드가 필요해지는 이유

여러 model request와 tool phase를 한 user turn 안에서 반복하면서 recovery·compact·budget 상태를 보존한다.

OBSERVED — queryLoop가 messages, turnCount, compaction/recovery 추적, pending summary, transition reason을 한 State로 소유한다.

INFERENCE — REPL이나 headless shell이 turn을 시작하지만, turn 내부의 model/tool/settle 상태기계는 queryLoop가 소유한다.

이 절에서 답할 결정 질문

명시적 State object는 여러 continue 경로에서 어떤 전이 누락을 막고, terminal reason은 어디에서 확정되는가?

이 설명을 확인할 코드 src/query.ts:252-321

ALGORITHM A10 · 선행 지식

문법 이름보다 먼저, 이 코드가 기대하는 실행 모델을 만든다

선행 지식은 용어 사전이 아니다. 아래 설명은 작은 실행 모델을 만든 다음, 그 모델이 src/query.ts의 결정점에서 어떤 오독을 막는지 되돌려 확인한다.

01
상태기계와 transition invariant

같은 while iteration이라도 현재 phase와 state에 따라 허용되는 다음 행동이 다르다. 전이는 일부 field만 바꾸는 assignment가 아니라 다음 phase가 믿을 수 있는 상태 전체를 만드는 일이다.

> **실제 코드로 돌아가면.** 252–321행의 초기 State가 이후 모든 continue branch에서 완전하게 갱신되는지 추적한다.

02
event sourcing과 현재 state의 차이

yield된 event 목록은 관찰 history이고 State는 다음 결정을 위한 현재 snapshot이다. 둘은 서로를 보완하지만 자동으로 같지 않다.

> **실제 코드로 돌아가면.** partial event를 냈지만 canonical messages가 아직 갱신되지 않은 구간을 구분한다.

이론은 정답이 아니라 코드에 던질 질문이다

상태·계산 모델
`state machine · transition invariant`
단계의 순서나 표현을 바꿔도 같은 상태와 불변조건에 도달하는가?

내구성·복구
`event sourcing`
중단 지점에서 재개했을 때 무엇을 재생하고 무엇을 다시 실행하지 않아야 하는가?

ALGORITHM A10 · 최소 예제

같은 continue라도 누락된 field 하나가 다음 phase의 의미를 바꾼다

전이 함수 없이 object를 부분 갱신할 때 pending 상태가 남는 경우를 축소해 본다.

설명을 위한 최소 실행 예제

```
let s = { phase: 'model', pendingTool: null, reason: null }
// model이 tool call을 냄
s = { ...s, phase: 'tool', pendingTool: 't1' }
// retry branch가 phase만 되돌림
s = { ...s, phase: 'model' }
// pendingTool === 't1'이 남아 다음 결과와 중복 pairing될 수 있음
```

관찰 시점	값과 상태	프로젝트 코드에서의 의미
초기	model phase, pending 없음	새 provider 응답을 받을 수 있다.
tool 전이	pendingTool=t1	tool result가 t1을 닫아야 한다.
불완전한 retry	phase만 model로 돌아감	과거 pending identity가 새 attempt에 남는다.

예제를 실제 구현에 다시 연결한다

실제 State의 각 continue site에서 transition reason과 pending/compact field가 함께 갱신되는지 확인하는 이유다.

ALGORITHM A10 · 실제 코드

OBSERVED

queryLoop 상태 전이의 결정점은 어디에 있는가

무한 while보다 중요한 것은 continue마다 새 State를 만드는 전이 규칙이다.

입력 표현. messages, system/user/system context, ToolUseContext, 그리고 limits

보존되는 작업 상태. messages, turnCount, compact tracking, max-output recovery count, pending summary, 그리고 transition reason

queryLoop state initialization

src/query.ts:252-321

```
252     // Immutable params — never reassigned during the query loop.
253     const {
254       systemPrompt,
255       userContext,
256       systemContext,
257       canUseTool,
258       fallbackModel,
259       querySource,
260       maxTurns,
261       skipCacheWrite,
262     } = params
263     const deps = params.deps ?? productionDeps()
264
265     // Mutable cross-iteration state. The loop body destructures this at the top
… 53 lines omitted; use the source link for the complete range …
319         stopHookActive,
320         turnCount,
321       } = state
```

이 발췌는 고정 commit의 실제 코드다. 다음 페이지에서는 문장 순서를 되풀이하지 않고, 각 branch가 상태의 의미를 어떻게 좁히는지 실행 전·중·후로 다시 읽는다.

ALGORITHM A10 · 상태 전이

호출 순서가 아니라, 판단 근거가 어떻게 변하는지를 추적한다

messages, system/user/system context, ToolUseContext, 그리고 limits은 처음부터 신뢰된 실행 상태가 아니다. messages, turnCount, compact tracking, max-output recovery count, pending summary, 그리고 transition reason가 앞선 관찰을 보존하기 때문에 뒤의 branch가 같은 작업을 계속 판단할 수 있다.

전제 정렬	진입 조건을 해석한다 immutable config를 snapshot한다. 이 결과가 다음 판단 "iteration state를 destructure한다."의 전제가 된다.
표현 정규화	표현과 선택지를 좁힌다 iteration state를 destructure한다. 이 결과가 다음 판단 "provider stream을 시작한다."의 전제가 된다.
유효성 판정	표현과 선택지를 좁힌다 provider stream을 시작한다. 이 결과가 다음 판단 "assistant blocks와 tool calls를 수집한다."의 전제가 된다.
결정 확정	표현과 선택지를 좁힌다 assistant blocks와 tool calls를 수집한다. 이 결과가 다음 판단 "필요하면 tools를 실행하고 message/state를 갱신한다."의 전제가 된다.
효과 조정	표현과 선택지를 좁힌다 필요하면 tools를 실행하고 message/state를 갱신한다. 이 결과가 다음 판단 "stop·compact·retry 조건에 따라 terminal 또는 다음 iteration으로 전이한다."의 전제가 된다.
상태 인계	후속 owner가 읽을 상태를 남긴다 stop·compact·retry 조건에 따라 terminal 또는 다음 iteration으로 전이한다. 이 결과를 다음 owner가 읽을 수 있을 때 이 절차가 닫힌다.

이 전이가 끝난 뒤 기대할 수 있는 조건

tool_use/tool_result pairing 없이 다음 provider call로 가지 않는다. 각 continue는 transition reason과 새 state를 남긴다. terminal path는 더 이상 effect를 시작하지 않는다.

ALGORITHM A10 · 설명의 보장 경계

이 구현이 지키려는 조건과, 설명이 멈춰야 하는 조건

여기서 불변조건은 형식적 증명의 결론이 아니다. 어느 전이가 의도한 의미를 보존했는지 읽고, 어떤 반례를 넣어 설명의 범위를 확인할지를 정하는 학습 도구다.

전이 전후에 확인할 조건

- **상태·계산 모델의 질문으로 읽기.** tool_use/tool_result pairing 없이 다음 provider call로 가지 않는다.
- **내구성·복구의 질문으로 읽기.** 각 continue는 transition reason과 새 state를 남긴다.
- **상태·계산 모델의 질문으로 읽기.** terminal path는 더 이상 effect를 시작하지 않는다.

최종 출력만 같아도 중간 identity나 effect가 달라질 수 있으므로, 각 조건을 보존하는 transition을 함께 관찰해야 한다.

설명의 범위를 드러내는 반례

- state field를 continue site 하나에서 누락
- partial message 뒤 retry로 중복 관찰
- stop hook가 terminal을 무한 지연

이 반례가 나타나면 “알고리즘이 틀렸다”로 끝내지 않고 어떤 state owner와 boundary에서 의미가 갈라졌는지 기록한다.

OBSERVED 명시적 State object로 여러 continue site의 partial assignment 위험을 줄인다.

RECOMMENDATION 전이 enum과 reducer를 추출하고 model/tool/settle phase를 property test한다.

ALGORITHM A10 · effect와 운영

정상 경로 뒤에 남는 effect와 실패 뒤에 남는 상태를 구분한다

model event를 소비하고 tool phase를 실행하며 budget·compact·stop 조건에 따라 다음 request 또는 terminal로 전이한다.

effect가 시작되고 관찰되는 경계

tool effect는 turn state와 분리된 외부 세계에 남을 수 있어 retry 전에 commit 여부를 알아야 한다.

현재 설계가 선택한 trade-off와 비용

명시적 State object로 여러 continue site의 partial assignment 위험을 줄인다. 이 선택의 계산·운영 비용은 iteration 수 T에 대해 O(T·(provider + tool + message projection)).로 기술되지만, 비용 표기만으로 중복 effect나 부분 상태의 의미가 해결되지는 않는다.

실패·중단 뒤에 실제로 남을 수 있는 상태

partial message 뒤 retry, stop hook의 반복 지연, continue site의 field 누락은 중복 관찰이나 무한 turn을 만든다. 이때 함께 대입할 반례는 state field를 continue site 하나에서 누락, partial message 뒤 retry로 중복 관찰, 그리고 stop hook가 terminal을 무한 지연이다. 이들을 모두 하나의 “실패”로 접으면 retry와 cleanup이 무엇을 이어받아야 하는지 알 수 없다.

관측이 다음 판단에 답하게 하는 법

phase, transition reason, attempt id, pending tool ids, terminal reason을 한 trace로 묶어야 한다. 로그의 존재보다 중요한 것은 이 관측값이 같은 요청·state·effect를 가리키며, 복구 또는 중단 결정을 실제로 바꿀 수 있는가이다.

ALGORITHM A10 · 현재 상태 장부

고정 snapshot에서 확인된 것과 아직 말할 수 없는 것을 함께 남긴다

이 장부는 앞의 내용을 압축한 요약이 아니라, 다음 독자가 현재 시스템을 다시 구성하기 위한 출발 상태다.

OBSERVED	명시적 State 초기화와 다수의 model/tool/compact 전이가 존재한다.
CONDITIONAL	모든 continue site가 필요한 field를 갱신할 때 다음 phase가 일관된 snapshot을 본다.
UNKNOWN	feature-gated 모든 전이와 production에서의 rare loop 조합은 이 발췌만으로 닫을 수 없다.
NEXT READ	reducer 추출 전 현재 continue site를 전이 표로 만들고 property test 입력으로 고정해야 한다.

다른 입력과 변경안에 전이해 볼 질문

1. partial assistant message 뒤 provider retry가 시작되면 새 attempt와 기존 event를 어떤 identity로 구분할 것인가?
2. stop hook가 다시 query를 요구할 때 turnCount와 terminal reason은 어느 전이에서 바뀌어야 하는가?

이 설명을 확인할 코드 src/query.ts:252-321

ALGORITHM A11 · 독자의 질문

tool-use/tool-result pairing 은 어떤 시스템 문제를 닫는가

모델은 tool_use를 요청하지만 registry에는 해당 tool이 없거나 permission이 거부될 수 있다. 그래도 provider protocol은 같은 id의 tool_result로 호출을 닫아야 다음 message를 해석할 수 있다.

코드가 필요해지는 이유

model이 요청한 tool call을 실행 가능한 tool과 연결하고 성공·거부·오류를 동일 protocol로 반환한다.

OBSERVED — toolExecution이 tool_use_id, request/message identity, progress, terminal result projection을 소유한다.

INFERENCE — 구체 tool은 effect를 소유하지만 orchestration layer가 protocol completion과 오류 변환을 소유한다.

이 절에서 답할 결정 질문

실행 성공·거부·unknown tool·오류를 어떻게 같은 correlation contract로 닫는가?

이 설명을 확인할 코드 src/services/tools/toolExecution.ts:337-410

ALGORITHM A11 · 선행 지식

문법 이름보다 먼저, 이 코드가 기대하는 실행 모델을 만든다

선행 지식은 용어 사전이 아니다. 아래 설명은 작은 실행 모델을 만든 다음, 그 모델이 src/services/tools/toolExecution.ts의 결정점에서 어떤 오독을 막는지 되돌려 확인한다.

01
correlation id와 protocol completion

비동기 요청과 결과는 도착 순서가 아니라 같은 id로 결합된다. 실행하지 못한 요청도 실패 result로 닫혀야 consumer가 pending 상태를 버릴 수 있다.

> **실제 코드로 돌아가면.** 337–410행에서 unknown tool branch도 어떤 id와 result shape를 반환하는지 본다.

이론은 정답이 아니라 코드에 던질 질문이다

프로토콜·식별
`correlation id · protocol completion`
생산자와 소비자가 동일한 값·작업·종료 의미를 해석한다는 증거가 있는가?

검증·관측·운영
`saga observation`
최종 출력뿐 아니라 실패·중단·복구 경로의 보장을 어떤 관찰과 테스트로 입증할 수 있는가?

ALGORITHM A11 · 실제 코드

OBSERVED

tool-use/tool-result pairing 의 결정점은 어디에 있는가

외부 effect는 둘 사이에서 일어나지만 대화 protocol에는 같은 id로 닫힌 원자가 필요하다.

입력 표현. ToolUseBlock id/name/input, assistant message, 그리고 tool registry

보존되는 작업 상태. tool id, request/message id, 그리고 progress와 result

unknown tool도 result로 닫기

src/services/tools/toolExecution.ts:337-410

```
337  export async function* runToolUse(
338    toolUse: ToolUseBlock,
339    assistantMessage: AssistantMessage,
340    canUseTool: CanUseToolFn,
341    toolUseContext: ToolUseContext,
342  ): AsyncGenerator<MessageUpdateLazy, void> {
343    const toolName = toolUse.name
344    // First try to find in the available tools (what the model sees)
345    let tool = findToolByName(toolUseContext.options.tools, toolName)
346
347    // If not found, check if it's a deprecated tool being called by alias
348    // (e.g., old transcripts calling "KillShell" which is now an alias for "TaskStop")
349    // Only fall back for tools where the name matches an alias, not the primary name
350    if (!tool) {
… 57 lines omitted; use the source link for the complete range …
408        }),
409      }
410      return
```

이 발췌는 고정 commit의 실제 코드다. 다음 페이지에서는 문장 순서를 되풀이하지 않고, 각 branch가 상태의 의미를 어떻게 좁히는지 실행 전·중·후로 다시 읽는다.

ALGORITHM A11 · 상태 전이

호출 순서가 아니라, 판단 근거가 어떻게 변하는지를 추적한다

ToolUseBlock id/name/input, assistant message, 그리고 tool registry은 처음부터 신뢰된 실행 상태가 아니다. tool id, request/message id, 그리고 progress와 result가 앞선 관찰을 보존하기 때문에 뒤의 branch가 같은 작업을 계속 판단할 수 있다.

전제 정렬	**진입 조건을 해석한다** tool name/alias를 resolve한다. 이 결과가 다음 판단 "없으면 동일 tool_use_id의 error result를 만든다."의 전제가 된다.
표현 정규화	**표현과 선택지를 좁힌다** 없으면 동일 tool_use_id의 error result를 만든다. 이 결과가 다음 판단 "abort를 확인한다."의 전제가 된다.
유효성 판정	**표현과 선택지를 좁힌다** abort를 확인한다. 이 결과가 다음 판단 "permission/execution pipeline을 호출한다."의 전제가 된다.
결정 확정	**표현과 선택지를 좁힌다** permission/execution pipeline을 호출한다. 이 결과가 다음 판단 "모든 terminal outcome을 tool_result message로 yield한다."의 전제가 된다.
상태 인계	**후속 owner가 읽을 상태를 남긴다** 모든 terminal outcome을 tool_result message로 yield한다. 이 결과를 다음 owner가 읽을 수 있을 때 이 절차가 닫힌다.

이 전이가 끝난 뒤 기대할 수 있는 조건

unknown tool도 silent drop하지 않는다. result는 원래 tool_use_id를 유지한다. 외부 effect 성공 여부와 protocol observation을 구분한다.

ALGORITHM A11 · 설명의 보장 경계

이 구현이 지키려는 조건과, 설명이 멈춰야 하는 조건

여기서 불변조건은 형식적 증명의 결론이 아니다. 어느 전이가 의도한 의미를 보존했는지 읽고, 어떤 반례를 넣어 설명의 범위를 확인할지를 정하는 학습 도구다.

전이 전후에 확인할 조건

- **프로토콜·식별의 질문으로 읽기.** unknown tool도 silent drop 하지 않는다.
- **검증·관측·운영의 질문으로 읽기.** result는 원래 tool_use_id를 유지한다.
- **프로토콜·식별의 질문으로 읽기.** 외부 effect 성공 여부와 protocol observation을 구분한다.

최종 출력만 같아도 중간 identity나 effect가 달라질 수 있으므로, 각 조건을 보존하는 transition을 함께 관찰해야 한다.

설명의 범위를 드러내는 반례

- effect는 발생했지만 result 기록 실패
- 같은 id 중복 result
- alias가 다른 tool identity로 해석

이 반례가 나타나면 “알고리즘이 틀렸다”로 끝내지 않고 어떤 state owner와 boundary에서 의미가 갈라졌는지 기록한다.

OBSERVED 예외를 loop 밖으로 유실하지 않고 model이 관찰 가능한 result로 변환한다.

RECOMMENDATION effect attempt id와 protocol tool_use_id를 함께 기록해 retry와 resume를 구분한다.

ALGORITHM A11 · effect와 운영

정상 경로 뒤에 남는 effect와 실패 뒤에 남는 상태를 구분한다

registry lookup 뒤 실행 또는 거부/오류를 model-visible tool_result로 투영한다.

effect가 시작되고 관찰되는 경계

실제 effect와 result journal은 별개 commit이므로 effect 성공 뒤 기록 실패가 가능하다.

현재 설계가 선택한 trade-off와 비용

예외를 loop 밖으로 유실하지 않고 model이 관찰 가능한 result로 변환한다. 이 선택의 계산·운영 비용은 tool lookup O(K), registry index가 없으면 tool 수 K에 선형.로 기술되지만, 비용 표기만으로 중복 effect나 부분 상태의 의미가 해결되지는 않는다.

실패·중단 뒤에 실제로 남을 수 있는 상태

중복 result, alias identity drift, effect 후 result 누락은 resume와 retry가 같은 작업을 다시 실행하게 할 수 있다. 이때 함께 대입할 반례는 effect는 발생했지만 result 기록 실패, 같은 id 중복 result, 그리고 alias가 다른 tool identity로 해석이다. 이들을 모두 하나의 "실패"로 접으면 retry와 cleanup이 무엇을 이어받아야 하는지 알 수 없다.

관측이 다음 판단에 답하게 하는 법

protocol id와 별도의 effect attempt id를 함께 기록해야 실행과 관찰을 구분할 수 있다. 로그의 존재보다 중요한 것은 이 관측값이 같은 요청·state·effect를 가리키며, 복구 또는 중단 결정을 실제로 바꿀 수 있는가이다.

ALGORITHM A11 · 현재 상태 장부

고정 snapshot에서 확인된 것과 아직 말할 수 없는 것을 함께 남긴다

이 장부는 앞의 내용을 압축한 요약이 아니라, 다음 독자가 현재 시스템을 다시 구성하기 위한 출발 상태다.

OBSERVED	unknown tool도 result로 닫는 경로와 tool id 연결이 존재한다.
CONDITIONAL	각 executor가 같은 completion contract를 지킬 때 provider context가 valid하게 유지된다.
UNKNOWN	외부 effect가 commit된 뒤 journal이 실패하는 모든 경우의 compensation은 확인되지 않는다.
NEXT READ	resume/retry 변경 시 tool_use_id 재사용과 effect attempt identity를 먼저 점검해야 한다.

다른 입력과 변경안에 전이해 볼 질문

1. 동일 tool_use_id의 result가 두 번 기록되면 어느 것을 canonical로 볼 것인가?
2. effect는 성공했지만 result write가 실패했을 때 retry가 무엇을 보고 중복 실행을 피할 수 있는가?

이 설명을 확인할 코드 src/services/tools/toolExecution.ts:337-410

SOURCE READING · 1/2

OBSERVED

Streaming은 텍스트가 아니라 typed event transport다

partial text, tool block, usage, stop, error는 같은 transport를 다른 consumer가 읽는다.

streaming query adapter

src/services/api/claude.ts:752-790

```
752   export async function* queryModelWithStreaming({
753     messages,
754     systemPrompt,
755     thinkingConfig,
756     tools,
757     signal,
758     options,
759   }: {
760     messages: Message[]
761     systemPrompt: SystemPrompt
762     thinkingConfig: ThinkingConfig
763     tools: Tools
764     signal: AbortSignal
765     options: Options
766   }): AsyncGenerator<
767     StreamEvent | AssistantMessage | SystemAPIErrorMessage,
768     void
769   > {
770     return yield* withStreamingVCR(messages, async function* () {
771       yield* queryModel(
772         messages,
773         systemPrompt,
774         thinkingConfig,
775         tools,
776         signal,
… 9 lines omitted; use the source link for the complete range …
786   function shouldDeferLspTool(tool: Tool): boolean {
787     if (!('isLsp' in tool) || !tool.isLsp) {
788       return false
789     }
790     const status = getInitializationStatus()
```

이 발췌는 설명을 위해 재작성한 의사코드가 아니라 고정 commit에서 기계적으로 가져온 실제 줄이다. 긴 범위는 가운데만 생략했으며 링크에서 전체 범위를 확인할 수 있다.

SOURCE READING · 2/2

OBSERVED

문법 아래에 숨은 상태와 책임

앞의 코드는 짧은 문법 표현을 사용하지만, 그 표현이 보장하는 범위와 프로젝트가 기대하는 범위는 같지 않다.

줄을 따라 확인할 관찰점

먼저 볼 표현	generator가 provider event를 yield한다.
그 표현이 만든 조건	AbortSignal과 options를 전달한다.
그 표현이 만든 조건	VCR/retry adapter가 stream을 감싼다.

언어와 런타임은 여기까지 보장한다

backpressure는 consumer의 next() 속도와 내부 buffering의 결합으로 생긴다.

프로젝트가 이 메커니즘에 맡긴 책임

UI는 partial text를 즉시 쓰지만 tool executor는 block completion을 기다려야 한다.

경계를 잘못 읽을 때 생기는 위험

한 projection이 partial을 저장하고 다른 projection이 버리면 resume 의미가 달라진다.

이 설명을 확인할 코드 src/services/api/claude.ts:752-790

DISTINCTION TABLE

OBSERVED

Turn의 네 phase

실행 순서가 필요한 곳만 phase로 표현한다.

Phase	입력	출력/commit
PREPARE	prompt·history·tools	provider request
MODEL	stream	assistant blocks
TOOLS	tool_use	effect + tool_result
SETTLE	usage·stop·journal	terminal result

이 표를 읽는 관점

각 phase 경계에는 identity와 이미 발생한 effect를 기록하는 commit point가 있다.

SELECTIVE EXECUTION TRACE

OBSERVED

사례의 첫 model/tool 왕복

config.ts 수정 요청이 첫 tool_use를 생성하는 순간을 추적한다.

책임 연결

MODEL

모델이 Read tool_use id=r1을 생성한다.

이때 남은 상태와 판단이 "TOOLS"에서 사용할 전제가 된다.

효과·위임

TOOLS

registry와 schema가 Read를 resolve한다.

이때 남은 상태와 판단이 "RESULT"에서 사용할 전제가 된다.

책임 연결

RESULT

내용과 file state가 tool_result id=r1로 돌아온다.

이때 남은 상태와 판단이 "MODEL"에서 사용할 전제가 된다.

책임 연결

MODEL

모델이 Edit tool_use id=e1을 생성한다.

이때 남은 상태와 판단이 "HANDOFF"에서 사용할 전제가 된다.

책임 연결

HANDOFF

4권의 effect pipeline으로 넘어간다.

이 지점에서 앞선 상태 변화가 외부에서 관찰 가능한 결과로 정산된다.

이 흐름 전체에서 보존해야 하는 것

r1과 e1은 서로 다른 correlation identity이며 Read 결과가 Edit의 낙관적 전제가 된다.

어디에서 설명이 틀어지는가

Read observation과 현재 file state 사이에 외부 변경이 생길 수 있다.

이 설명을 확인할 코드 src/tools/FileReadTool/FileReadTool.ts:337-380 src/tools/FileEditTool/FileEditTool.ts:86-130

05 Provider stream을 canonical message로 재구성한다

API transport는 text chunk를 전달하는 얇은 pipe가 아니다. 부분 content block, 늦게 도착하는 usage와 stop reason, watchdog, request identity를 하나의 내부 message protocol로 조립한다.

이 장의 논증은 다음 질문을 차례로 해결한다

코드	의미 연결 queryModelWithStreaming은 provider adapter를 감싸는 protocol 입구다	알고리즘	책임 연결 부분 content block의 typed reconstruction
코드	책임 연결 message_delta는 이미 yield된 message의 마지막 metadata를 정산한다	코드	책임 연결 stream idle watchdog는 중단과 정상 완료를 구분한다
구분	책임 연결 Provider event와 내부 상태의 합류 시점	사례	진입 경계 tool_use JSON이 실행 가능한 input이 되기까지

코드 독해의 시작점

`src/services/api/claude.ts` → `src/services/api/withRetry.ts` → `src/types/message.ts`

각 절은 앞 절의 결론을 다음 절의 전제로 사용한다. 따라서 용어 목록을 외우기보다, 현재 구현이 어떤 상태를 만들고 그 상태가 다음 결정의 조건으로 어떻게 사용되는지 연결해서 읽는다.

SOURCE READING · 1/2

OBSERVED

queryModelWithStreaming은 provider adapter를 감싸는 protocol 입구다

짧은 wrapper는 messages·system prompt·thinking·tools·AbortSignal·options를 같은 attempt context로 묶고 VCR/retry 경계를 통과시킨다.

streaming model adapter

src/services/api/claude.ts:752-774

```
752  export async function* queryModelWithStreaming({
753    messages,
754    systemPrompt,
755    thinkingConfig,
756    tools,
757    signal,
758    options,
759  }: {
760    messages: Message[]
761    systemPrompt: SystemPrompt
762    thinkingConfig: ThinkingConfig
763    tools: Tools
764    signal: AbortSignal
765    options: Options
766  }): AsyncGenerator<
767    StreamEvent | AssistantMessage | SystemAPIErrorMessage,
768    void
769  > {
770    return yield* withStreamingVCR(messages, async function* () {
771      yield* queryModel(
772        messages,
773        systemPrompt,
774        thinkingConfig,
```

> 이 발췌는 설명을 위해 재작성한 의사코드가 아니라 고정 commit에서 기계적으로 가져온 실제 줄이다. 긴 범위는 가운데만 생략했으며 링크에서 전체 범위를 확인할 수 있다.

SOURCE READING · 2/2

OBSERVED

문법 아래에 숨은 상태와 책임

앞의 코드는 짧은 문법 표현을 사용하지만, 그 표현이 보장하는 범위와 프로젝트가 기대하는 범위는 같지 않다.

줄을 따라 확인할 관찰점

먼저 볼 표현	반환 타입은 provider event뿐 아니라 canonical AssistantMessage와 SystemAPIErrorMessage를 포함한다.
그 표현이 만든 조건	yield*는 inner queryModel의 중간 event를 전달하지만 wrapper 자체의 별도 message state를 만들지 않는다.
그 표현이 만든 조건	AbortSignal과 model/options가 stream lifetime 전체에 고정된다.

언어와 런타임은 여기까지 보장한다

AsyncGenerator union은 transport event와 domain message가 시간 순서로 섞일 수 있음을 타입으로 드러낸다. 그러나 union만으로 각 variant의 순서나 pairing이 보장되지는 않으므로 consumer는 discriminant와 lifecycle을 함께 해석해야 한다.

프로젝트가 이 메커니즘에 맡긴 책임

이 entry는 provider SDK, retry/VCR, query loop 사이의 anti-corruption boundary다. model call을 Promise 하나로 감추지 않고 stream observation을 보존해 UI·telemetry·tool loop가 서로 다른 projection을 만들 수 있게 한다.

경계를 잘못 읽을 때 생기는 위험

consumer가 StreamEvent와 AssistantMessage를 같은 durable record로 취급하거나, retry wrapper가 이미 노출된 event를 새 attempt와 구별하지 않으면 중복 text와 잘못된 tool identity가 생긴다.

이 설명을 확인할 코드 src/services/api/claude.ts:752-774

ALGORITHM A39 · 독자의 질문

부분 content block의 typed reconstruction은 어떤 시스템 문제를 닫는가

provider는 완성 message가 아니라 message_start, 여러 index의 block start/delta/stop, 늦은 message_delta를 보낸다. UI에는 partial content를 즉시 보여 주면서 transcript에는 protocol-valid한 canonical message를 남겨야 한다.

코드가 필요해지는 이유

text, thinking, tool input JSON, connector block이 서로 다른 delta로 도착하는 동안 type과 index를 잃지 않고 내부 message를 만든다.

OBSERVED — claude.ts의 stream consumer가 partialMessage, contentBlocks, usage, stopReason, newMessages와 variant별 상태를 소유한다.

INFERENCE — provider SDK는 event를 생산하지만 local consumer가 canonical block builder를 소유하고 UI/transcript는 그 projection을 소비한다.

이 절에서 답할 결정 질문

index별 mutable accumulator와 object identity가 partial visibility, tool JSON 완성, late metadata 정산을 어떻게 동시에 보존하는가?

이 설명을 확인할 코드 src/services/api/claude.ts:1940-2054 src/services/api/claude.ts:2053-2210 src/services/api/claude.ts:2211-2302

ALGORITHM A39 · 선행 지식

문법 이름보다 먼저, 이 코드가 기대하는 실행 모델을 만든다

선행 지식은 용어 사전이 아니다. 아래 설명은 작은 실행 모델을 만든 다음, 그 모델이 src/services/api/claude.ts의 결정점에서 어떤 오독을 막는지 되돌려 확인한다.

01 discriminated union과 runtime validation

type tag는 각 delta가 어떤 accumulator variant와 결합할 수 있는지 code branch를 좁힌다. 하지만 network가 start-before-delta 순서를 지킨다는 사실은 TypeScript가 보장하지 않으므로 runtime check가 필요하다.

> **실제 코드로 돌아가면.** 1940–2054행에서 index lookup, type 비교, missing block error가 함께 나타나는 지점을 본다.

02 mutable builder와 referential identity

stream 중에는 조각을 한 object에 누적하고 늦은 metadata를 같은 reference에 반영한다. immutable replacement는 안전해 보이지만 이미 그 reference를 잡은 지연 serializer와 identity를 갈라놓을 수 있다.

> **실제 코드로 돌아가면.** block stop 이후 yield와 message_delta mutation, transcript queue serialization timing을 함께 추적한다.

이론은 정답이 아니라 코드에 던질 질문이다

상태·계산 모델
`state machine`
단계의 순서나 표현을 바꿔도 같은 상태와 불변조건에 도달하는가?

프로토콜·식별
`tagged union · referential integrity`
생산자와 소비자가 동일한 값·작업·종료 의미를 해석한다는 증거가 있는가?

ALGORITHM A39 · 최소 예제

보이는 완료와 metadata의 terminal 완료는 같은 시점이 아니다

text block stop 뒤 usage가 늦게 오는 event sequence를 축소한다.

설명을 위한 최소 실행 예제

```
start(message)
startBlock(0, 'text')
delta(0, '안녕')
stopBlock(0)       // content를 yield
messageDelta({ usage: 12, stop: 'end_turn' }) // 같은 message identity 갱신
```

관찰 시점	값과 상태	프로젝트 코드에서의 의미
block start	index 0 builder 생성	delta를 받을 valid state가 열린다.
block stop	text content가 화면에 보임	message 전체 metadata는 아직 terminal이 아니다.
message delta	usage/stopReason 갱신	지연 transcript serializer가 같은 identity를 봐야 한다.

예제를 실제 구현에 다시 연결한다

yield 직후 deep copy나 object replacement가 late metadata를 잃게 할 수 있는 이유를 실제 queue와 함께 읽는다.

ALGORITHM A39 · 실제 코드

OBSERVED

부분 content block의 typed reconstruction의 결정점은 어디에 있는가

provider event sequence를 index별 mutable builder에 누적하고 block completion에서 immutable-looking AssistantMessage로 투영한다.

입력 표현. message_start, content_block_start/delta/stop, message_delta, 그리고 tool registry와 request id

보존되는 작업 상태. partialMessage, index별 contentBlocks, usage, stopReason, newMessages, 그리고 research/advisor state

message start와 content block 초기화

src/services/api/claude.ts:1940-2054

```
1940        for await (const part of stream) {
1941          resetStreamIdleTimer()
1942          const now = Date.now()
1943
1944          // Detect and log streaming stalls (only after first event to avoid counting TTFB)
1945          if (lastEventTime !== null) {
1946            const timeSinceLastEvent = now - lastEventTime
1947            if (timeSinceLastEvent > STALL_THRESHOLD_MS) {
1948              stallCount++
1949              totalStallTime += timeSinceLastEvent
1950              logForDebugging(
1951                `Streaming stall detected: ${(timeSinceLastEvent / 1000).toFixed(1)}s gap between events
(stall #${stallCount})`,
1952                { level: 'warn' },
1953              )
… 98 lines omitted; use the source link for the complete range …
2052              break
2053            case 'content_block_delta': {
2054              const contentBlock = contentBlocks[part.index]
```

이 발췌는 고정 commit의 실제 코드다. 다음 페이지에서는 문장 순서를 되풀이하지 않고, 각 branch가 상태의 의미를 어떻게 좁히는지 실행 전·중·후로 다시 읽는다.

ALGORITHM A39 · 상태 전이

호출 순서가 아니라, 판단 근거가 어떻게 변하는지를 추적한다

message_start, content_block_start/delta/stop, message_delta, 그리고 tool registry와 request id은 처음부터 신뢰된 실행 상태가 아니다. partialMessage, index별 contentBlocks, usage, stopReason, newMessages, 그리고 research/advisor state가 앞선 관찰을 보존하기 때문에 뒤의 branch가 같은 작업을 계속 판단할 수 있다.

전제 정렬	진입 조건을 해석한다 message_start에서 envelope와 초기 usage를 잡는다. 이 결과가 다음 판단 "content_block_start에서 variant별 빈 accumulator를 만든다."의 전제가 된다.
표현 정규화	표현과 선택지를 좁힌다 content_block_start에서 variant별 빈 accumulator를 만든다. 이 결과가 다음 판단 "delta discriminant와 기존 block type이 맞는지 검사한 뒤 text·JSON·thinking을 누적한다."의 전제가 된다.
유효성 판정	표현과 선택지를 좁힌다 delta discriminant와 기존 block type이 맞는지 검사한 뒤 text·JSON·thinking을 누적한다. 이 결과가 다음 판단 "content_block_stop에서 해당 block을 normalize해 AssistantMessage를 만들고 yield한다."의 전제가 된다.
결정 확정	표현과 선택지를 좁힌다 content_block_stop에서 해당 block을 normalize해 AssistantMessage를 만들고 yield한다. 이 결과가 다음 판단 "message_delta에서 최종 usage와 stop reason을 이미 만든 마지막 message에 반영한다."의 전제가 된다.
효과 조정	표현과 선택지를 좁힌다 message_delta에서 최종 usage와 stop reason을 이미 만든 마지막 message에 반영한다. 이 결과가 다음 판단 "원본 provider event도 stream_event로 별도 yield한다."의 전제가 된다.
상태 인계	후속 owner가 읽을 상태를 남긴다 원본 provider event도 stream_event로 별도 yield한다. 이 결과를 다음 owner가 읽을 수 있을 때 이 절차가 닫힌다.

이 전이가 끝난 뒤 기대할 수 있는 조건

delta index에는 먼저 시작된 block이 존재한다. delta type과 accumulator variant가 일치한다. tool input JSON은 block 완료 전까지 문자열 조각이며 실행 input으로 사용되지 않는다. 최종 usage와 stop reason이 transcript가 직렬화할 같은 object reference에 반영된다.

ALGORITHM A39 · 설명의 보장 경계

이 구현이 지키려는 조건과, 설명이 멈춰야 하는 조건

여기서 불변조건은 형식적 증명의 결론이 아니다. 어느 전이가 의도한 의미를 보존했는지 읽고, 어떤 반례를 넣어 설명의 범위를 확인할지를 정하는 학습 도구다.

전이 전후에 확인할 조건

- **상태·계산 모델의 질문으로 읽기.** delta index에는 먼저 시작된 block이 존재한다.
- **프로토콜·식별의 질문으로 읽기.** delta type과 accumulator variant가 일치한다.
- **상태·계산 모델의 질문으로 읽기.** tool input JSON은 block 완료 전까지 문자열 조각이며 실행 input으로 사용되지 않는다.
- **프로토콜·식별의 질문으로 읽기.** 최종 usage와 stop reason이 transcript가 직렬화할 같은 object reference에 반영된다.

최종 출력만 같아도 중간 identity나 effect가 달라질 수 있으므로, 각 조건을 보존하는 transition을 함께 관찰해야 한다.

설명의 범위를 드러내는 반례

- start 없는 delta
- index 재사용
- partial JSON을 조기 parse
- message_delta 이전에 object reference를 복제해 최종 metadata 유실

이 반례가 나타나면 "알고리즘이 틀렸다"로 끝내지 않고 어떤 state owner와 boundary에서 의미가 갈라졌는지 기록한다.

OBSERVED provider SDK의 mutable/중복 동작을 그대로 퍼뜨리지 않고 local accumulator와 explicit mismatch error로 정규화한다.

RECOMMENDATION block builder를 variant별 reducer로 분리하고 event sequence property test로 start→delta*→stop 규약과 malformed 순서를 검증한다.

ALGORITHM A39 · effect와 운영

정상 경로 뒤에 남는 effect와 실패 뒤에 남는 상태를 구분한다

event마다 block map을 갱신하고 완료 block/message를 yield하며 idle watchdog를 reset한다.

effect가 시작되고 관찰되는 경계

UI projection, transcript queue, telemetry timer가 서로 다른 시점에 같은 message를 관찰한다.

현재 설계가 선택한 trade-off와 비용

provider SDK의 mutable/중복 동작을 그대로 퍼뜨리지 않고 local accumulator와 explicit mismatch error로 정규화한다. 이 선택의 계산·운영 비용은 event 수 E와 누적 payload 크기 P에 O(E + P). 문자열 반복 결합은 payload와 engine 구현에 따라 추가 복사 비용이 생길 수 있다.로 기술되지만, 비용 표기만으로 중복 effect나 부분 상태의 의미가 해결되지는 않는다.

실패·중단 뒤에 실제로 남을 수 있는 상태

start 없는 delta, index 재사용, partial JSON 조기 parse, stale reference, watchdog cleanup 누락은 서로 다른 불완전 상태를 만든다. 이때 함께 대입할 반례는 start 없는 delta, index 재사용, partial JSON을 조기 parse, 그리고 message_delta 이전에 object reference를 복제해 최종 metadata 유실이다. 이들을 모두 하나의 “실패”로 접으면 retry와 cleanup이 무엇을 이어받아야 하는지 알 수 없다.

관측이 다음 판단에 답하게 하는 법

request id, event index/type, builder state, yield identity, terminal metadata, timeout reason을 연결해야 한다. 로그의 존재보다 중요한 것은 이 관측값이 같은 요청·state·effect를 가리키며, 복구 또는 중단 결정을 실제로 바꿀 수 있는가이다.

ALGORITHM A39 · 현재 상태 장부

고정 snapshot에서 확인된 것과 아직 말할 수 없는 것을 함께 남긴다

이 장부는 앞의 내용을 압축한 요약이 아니라, 다음 독자가 현재 시스템을 다시 구성하기 위한 출발 상태다.

OBSERVED	typed block accumulator, missing-block runtime error, completion yield, late metadata mutation, raw event projection, idle watchdog가 존재한다.
CONDITIONAL	provider sequence와 local variant checks가 일치하고 serializer가 같은 object identity를 볼 때 canonical message가 완성된다.
UNKNOWN	malformed sequence의 production 빈도, feature-gated connector/advisor 활성화, 모든 runtime의 queue timing은 확인되지 않는다.
NEXT READ	variant별 reducer와 event sequence property test를 도입하기 전 1940–2302행과 transcript serializer timing을 한 trace로 재검증해야 한다.

다른 입력과 변경안에 전이해 볼 질문

1. stopBlock 전에 partial tool JSON을 parse하면 어떤 invalid input이 실행 가능한 값처럼 보이는가?
2. message object를 immutable replacement하면 이미 queue에 들어간 reference의 usage와 stopReason은 어떻게 되는가?

이 설명을 확인할 코드 src/services/api/claude.ts:1940-2054 src/services/api/claude.ts:2053-2210 src/services/api/claude.ts:2211-2302

SOURCE READING · 1/2

OBSERVED

message_delta는 이미 yield된 message의 마지막 metadata를 정산한다

content block이 먼저 consumer에게 보인 뒤 usage와 stop reason이 도착하므로, 완료란 새 object를 반환하는 일이 아니라 공유 reference의 마지막 필드를 확정하는 과정이 된다.

late metadata mutation

src/services/api/claude.ts:2211-2251

```
2211              break
2212            }
2213            case 'message_delta': {
2214              usage = updateUsage(usage, part.usage)
2215              // Capture research from message_delta if available (internal only).
2216              // Always overwrite with the latest value. Also write back to
2217              // already-yielded messages since message_delta arrives after
2218              // content_block_stop.
2219              if (
2220                process.env.USER_TYPE === 'ant' &&
2221                'research' in (part as unknown as Record<string, unknown>)
2222              ) {
2223                research = (part as unknown as Record<string, unknown>).research
2224                for (const msg of newMessages) {
2225                  msg.research = research
2226                }
2227              }
2228
2229              // Write final usage and stop_reason back to the last yielded
2230              // message. Messages are created at content_block_stop from
2231              // partialMessage, which was set at message_start before any tokens
2232              // were generated (output_tokens: 0, stop_reason: null).
2233              // message_delta arrives after content_block_stop with the real
2234              // values.
… 12 lines omitted; use the source link for the complete range …
2247                lastMsg.message.stop_reason = stopReason
2248              }
2249
2250              // Update cost
2251              const costUSDForPart = calculateUSDCost(resolvedModel, usage)
```

이 발췌는 설명을 위해 재작성한 의사코드가 아니라 고정 commit에서 기계적으로 가져온 실제 줄이다. 긴 범위는 가운데만 생략했으며 링크에서 전체 범위를 확인할 수 있다.

SOURCE READING · 2/2

OBSERVED

문법 아래에 숨은 상태와 책임

앞의 코드는 짧은 문법 표현을 사용하지만, 그 표현이 보장하는 범위와 프로젝트가 기대하는 범위는 같지 않다.

줄을 따라 확인할 관찰점

먼저 볼 표현	message_delta가 usage를 합산하고 stopReason을 읽는다.
그 표현이 만든 조건	newMessages의 마지막 message는 content_block_stop에서 이미 yield되었다.
그 표현이 만든 조건	transcript write queue는 message.message reference를 나중에 직렬화한다.
그 표현이 만든 조건	object replacement가 아니라 direct property mutation을 사용한다.

언어와 런타임은 여기까지 보장한다

JavaScript object는 reference identity를 공유한다. queue가 object를 즉시 복사하지 않고 나중에 읽는다면 같은 object의 field mutation은 보이지만 새 object로 local variable만 교체한 결과는 보이지 않는다. 이 동작은 타입이 아니라 aliasing과 serialization 시점에서 나온다.

프로젝트가 이 메커니즘에 맡긴 책임

프로젝트는 late provider metadata와 지연 transcript flush 사이의 시간차를 같은 reference로 연결한다. 이는 일반적인 immutability 규칙의 예외이며, 왜 예외가 필요한지를 주석과 test로 보존해야 하는 지점이다.

경계를 잘못 읽을 때 생기는 위험

flush timing이 바뀌어 더 일찍 serialize하거나 다른 consumer가 yield 직후 deep copy하면 usage와 stop reason이 서로 다른 projection에서 달라진다. 반대로 mutation이 너무 늦으면 이미 영구 기록된 값은 고쳐지지 않는다.

이 설명을 확인할 코드 src/services/api/claude.ts:2211-2251

SOURCE READING · 1/2

OBSERVED

stream idle watchdog는 중단과 정상 완료를 구분한다

chunk가 오지 않는 stream을 무한 대기로 두지 않고 warning과 abort를 별도 timer로 관리한 뒤 non-streaming recovery로 넘긴다.

idle watchdog와 stall observation

src/services/api/claude.ts:1880-1949

```
1880      let streamIdleAborted = false
1881      // performance.now() snapshot when watchdog fires, for measuring abort propagation delay
1882      let streamWatchdogFiredAt: number | null = null
1883      let streamIdleWarningTimer: ReturnType<typeof setTimeout> | null = null
1884      let streamIdleTimer: ReturnType<typeof setTimeout> | null = null
1885      function clearStreamIdleTimers(): void {
1886        if (streamIdleWarningTimer !== null) {
1887          clearTimeout(streamIdleWarningTimer)
1888          streamIdleWarningTimer = null
1889        }
1890        if (streamIdleTimer !== null) {
1891          clearTimeout(streamIdleTimer)
1892          streamIdleTimer = null
1893        }
1894      }
1895      function resetStreamIdleTimer(): void {
1896        clearStreamIdleTimers()
1897        if (!streamWatchdogEnabled) {
1898          return
1899        }
1900        streamIdleWarningTimer = setTimeout(
1901          warnMs => {
1902            logForDebugging(
1903              `Streaming idle warning: no chunks received for ${warnMs / 1000}s`,
1904              { level: 'warn' },
… 40 lines omitted; use the source link for the complete range …
1945          if (lastEventTime !== null) {
1946            const timeSinceLastEvent = now - lastEventTime
1947            if (timeSinceLastEvent > STALL_THRESHOLD_MS) {
1948              stallCount++
1949              totalStallTime += timeSinceLastEvent
```

> 이 발췌는 설명을 위해 재작성한 의사코드가 아니라 고정 commit에서 기계적으로 가져온 실제 줄이다. 긴 범위는 가운데만 생략했으며 링크에서 전체 범위를 확인할 수 있다.

SOURCE READING · 2/2

OBSERVED

문법 아래에 숨은 상태와 책임

앞의 코드는 짧은 문법 표현을 사용하지만, 그 표현이 보장하는 범위와 프로젝트가 기대하는 범위는 같지 않다.

줄을 따라 확인할 관찰점

먼저 볼 표현	새 chunk마다 warning/timeout timer를 reset한다.
그 표현이 만든 조건	첫 chunk 전 대기는 TTFB이고 이후 gap은 stall로 분리한다.
그 표현이 만든 조건	timeout은 streamIdleAborted와 firing timestamp를 남긴 뒤 resource release를 호출한다.
그 표현이 만든 조건	loop 종료 뒤 watchdog abort인지 확인해 정상 completion과 다른 recovery를 택한다.

언어와 런타임은 여기까지 보장한다

timer callback은 stream iterator와 별도 event-loop task다. boolean과 timestamp는 두 실행 경로가 합류할 때 terminal 원인을 전달하는 handoff state이며, timer cleanup은 이미 끝난 stream에 뒤늦은 abort가 들어오지 않게 한다.

프로젝트가 이 메커니즘에 맡긴 책임

이 코드는 latency telemetry, liveness, abort propagation, fallback transport를 하나의 terminal classification으로 연결한다. 단순 timeout wrapper보다 "왜 loop가 끝났는가"를 보존하는 것이 핵심이다.

경계를 잘못 읽을 때 생기는 위험

timer를 reset하거나 clear하지 않으면 정상 stream을 abort할 수 있고, watchdog abort를 정상 EOF로 취급하면 partial message를 성공으로 정산한다. warning/stall telemetry가 effect control과 같은 flag를 바꾸면 관찰 코드가 동작을 왜곡한다.

이 설명을 확인할 코드 src/services/api/claude.ts:1880-1949

DISTINCTION TABLE

OBSERVED

Provider event와 내부 상태의 합류 시점

각 event는 화면에 보이는 시점과 canonical message가 확정되는 시점이 다르다.

Provider event	유지되는 working state	외부에서 말할 수 있는 것
message_start	partialMessage·initial usage	request envelope가 시작됨
content_block_start	typed empty accumulator	block variant와 index가 정해짐
content_block_delta	text·partial JSON·thinking 누적	부분 관찰만 가능
content_block_stop	normalized block→AssistantMessage	한 block message를 yield 가능
message_delta	final usage·stop reason mutation	비용·terminal metadata 확정
message_stop	stream loop 종료 조건	transport sequence 종료

이 표를 읽는 관점

"텍스트가 보였다"와 "message가 terminal metadata까지 정산되었다"를 같은 완료로 부르면 transcript, 비용, retry 판단이 갈라진다.

SELECTIVE EXECUTION TRACE

OBSERVED

tool_use JSON이 실행 가능한 input이 되기까지

partial_json이 들어오는 경우만 골라 transport와 effect 사이의 경계를 본다.

책임 연결

Start

tool_use block을 input: 빈 문자열 accumulator로 만든다.

이때 남은 상태와 판단이 "Delta"에서 사용할 전제가 된다.

책임 연결

Delta

같은 index와 tool_use variant인지 확인하고 partial_json을 이어 붙인다.

이때 남은 상태와 판단이 "Stop"에서 사용할 전제가 된다.

책임 연결

Stop

완성된 문자열을 provider content block으로 normalize한다.

이때 남은 상태와 판단이 "Message"에서 사용할 전제가 된다.

책임 연결

Message

tool_use id와 name을 가진 AssistantMessage를 yield한다.

이때 남은 상태와 판단이 "Query loop"에서 사용할 전제가 된다.

책임 연결

Query loop

완료된 block만 Tool schema parse와 permission pipeline에 넘긴다.

이 지점에서 앞선 상태 변화가 외부에서 관찰 가능한 결과로 정산된다.

이 흐름 전체에서 보존해야 하는 것

partial transport bytes는 effect authority를 갖지 않으며 block completion, JSON parse, Tool schema, permission을 모두 통과해야 한다.

어디에서 설명이 틀어지는가

중간 JSON이 우연히 parse된다는 이유로 실행하면 뒤 delta가 의미를 바꾸거나 다른 tool input으로 완성될 수 있다.

이 설명을 확인할 코드 src/services/api/claude.ts:2053-2125 src/services/tools/toolExecution.ts:337-410

06

Retry, budget, compaction 은 의미를 바꾼다

운영 장치는 단순 최적화가 아니라 어떤 과거와 effect를 다음 iteration이 보는지 결정한다.

이 장의 논증은 다음 질문을 차례로 해결한다

알고리즘	책임 연결 API retry와 backoff	알고리즘	기록·복구 Compaction as lossy migration
코드	책임 연결 Budget check 위치가 policy 의미를 만든다	구분	책임 연결 Retry 가능성 판정
알고리즘	책임 연결 Message normalization lookup 구축	사례	책임 연결 3권에서 보존한 사례 상태

코드 독해의 시작점

`src/services/api/withRetry.ts` → `src/services/compact/**` → `src/query/tokenBudget.ts`

각 절은 앞 절의 결론을 다음 절의 전제로 사용한다. 따라서 용어 목록을 외우기보다, 현재 구현이 어떤 상태를 만들고 그 상태가 다음 결정의 조건으로 어떻게 사용되는지 연결해서 읽는다.

ALGORITHM A12 · 독자의 질문

API retry와 backoff은 어떤 시스템 문제를 닫는가

provider 오류를 다시 시도하면 가용성은 높아지지만, partial output이나 tool request가 이미 관찰된 뒤라면 같은 요청의 반복이 아니다.

코드가 필요해지는 이유

일시적 provider 실패에서 cache·latency·rate limit을 고려해 요청을 재시도한다.

OBSERVED withRetry가 attempt, retryContext, fastMode, lastError와 AbortSignal 반응을 소유한다.

INFERENCE retry wrapper는 provider call을 감싸지만 tool effect의 idempotency까지 자동으로 보장하지 않는다.

이 절에서 답할 결정 질문

retry 가능 오류, attempt budget, 첫 visible event, external commit point를 어떻게 함께 판단하는가?

이 설명을 확인할 코드 src/services/api/withRetry.ts:170-220 src/services/api/withRetry.ts:430-516

ALGORITHM A12 · 선행 지식

문법 이름보다 먼저, 이 코드가 기대하는 실행 모델을 만든다

선행 지식은 용어 사전이 아니다. 아래 설명은 작은 실행 모델을 만든 다음, 그 모델이 src/services/api/withRetry.ts의 결정점에서 어떤 오독을 막는지 되돌려 확인한다.

01

지수 backoff와 retry budget

실패가 반복될수록 대기를 늘려 동시 재시도 폭주를 줄인다. 그러나 횟수 제한만으로 이미 관찰된 결과나 commit된 effect의 중복을 막지는 못한다.

> **실제 코드로 돌아가면.** 170–220행에서 attempt 증가, abort, foreground/background 정책, 모델 전환을 따로 읽는다.

이론은 정답이 아니라 코드에 던질 질문이다

동시성·시간·합의
`exponential backoff · retry budget`
stale read, 중복 실행, 취소가 끼어들어도 effect와 최종 상태의 의미가 보존되는가?

내구성·복구
`idempotency/commit point`
중단 지점에서 재개했을 때 무엇을 재생하고 무엇을 다시 실행하지 않아야 하는가?

ALGORITHM A12 · 실제 코드

OBSERVED

API retry와 backoff의 결정점은 어디에 있는가

retry 여부는 status code뿐 아니라 source, fast mode, retry-after, consumer-visible event에 좌우된다.

입력 표현. operation, maxRetries, AbortSignal, query source, 그리고 model/thinking config

보존되는 작업 상태. attempt, retryContext, fastMode, 그리고 lastError

retry context와 attempt loop

src/services/api/withRetry.ts:170-220

```
170  export async function* withRetry<T>(
171    getClient: () => Promise<Anthropic>,
172    operation: (
173      client: Anthropic,
174      attempt: number,
175      context: RetryContext,
176    ) => Promise<T>,
177    options: RetryOptions,
178  ): AsyncGenerator<SystemAPIErrorMessage, T> {
179    const maxRetries = getMaxRetries(options)
180    const retryContext: RetryContext = {
181      model: options.model,
182      thinkingConfig: options.thinkingConfig,
183      ...(isFastModeEnabled() && { fastMode: options.fastMode }),
… 34 lines omitted; use the source link for the complete range …
218        const isStaleConnection = isStaleConnectionError(lastError)
219        if (
220          isStaleConnection &&
```

이 발췌는 고정 commit의 실제 코드다. 다음 페이지에서는 문장 순서를 되풀이하지 않고, 각 branch가 상태의 의미를 어떻게 좁히는지 실행 전·중·후로 다시 읽는다.

ALGORITHM A12 · 상태 전이

호출 순서가 아니라, 판단 근거가 어떻게 변하는지를 추적한다

operation, maxRetries, AbortSignal, query source, 그리고 model/thinking config은 처음부터 신뢰된 실행 상태가 아니다. attempt, retryContext, fastMode, 그리고 lastError가 앞선 관찰을 보존하기 때문에 뒤의 branch가 같은 작업을 계속 판단할 수 있다.

전제 정렬	**진입 조건을 해석한다** client와 context를 준비한다. 이 결과가 다음 판단 "operation을 시도한다."의 전제가 된다.
표현 정규화	**표현과 선택지를 좁힌다** operation을 시도한다. 이 결과가 다음 판단 "오류 종류와 source를 분류한다."의 전제가 된다.
유효성 판정	**표현과 선택지를 좁힌다** 오류 종류와 source를 분류한다. 이 결과가 다음 판단 "retry-after 또는 지수 backoff를 계산한다."의 전제가 된다.
결정 확정	**표현과 선택지를 좁힌다** retry-after 또는 지수 backoff를 계산한다. 이 결과가 다음 판단 "긴 대기에는 keep-alive event를 yield한다."의 전제가 된다.
효과 조정	**표현과 선택지를 좁힌다** 긴 대기에는 keep-alive event를 yield한다. 이 결과가 다음 판단 "limit 도달 시 CannotRetryError로 종료한다."의 전제가 된다.
상태 인계	**후속 owner가 읽을 상태를 남긴다** limit 도달 시 CannotRetryError로 종료한다. 이 결과를 다음 owner가 읽을 수 있을 때 이 절차가 닫힌다.

이 전이가 끝난 뒤 기대할 수 있는 조건

abort는 새 retry를 시작하지 않는다. non-foreground source는 capacity cascade를 증폭하지 않는다. 이미 노출한 partial/effect가 있다면 단순 동일 요청으로 취급하지 않는다.

ALGORITHM A12 · 설명의 보장 경계

이 구현이 지키려는 조건과, 설명이 멈춰야 하는 조건

여기서 불변조건은 형식적 증명의 결론이 아니다. 어느 전이가 의도한 의미를 보존했는지 읽고, 어떤 반례를 넣어 설명의 범위를 확인할지를 정하는 학습 도구다.

전이 전후에 확인할 조건

- **동시성·시간·합의의 질문으로 읽기.** abort는 새 retry를 시작하지 않는다.
- **내구성·복구의 질문으로 읽기.** non-foreground source는 capacity cascade를 증폭하지 않는다.
- **동시성·시간·합의의 질문으로 읽기.** 이미 노출한 partial/effect가 있다면 단순 동일 요청으로 취급하지 않는다.

최종 출력만 같아도 중간 identity나 effect가 달라질 수 있으므로, 각 조건을 보존하는 transition을 함께 관찰해야 한다.

설명의 범위를 드러내는 반례

- partial output 중복
- tool_use 뒤 provider retry
- retry storm
- cache model switch

이 반례가 나타나면 "알고리즘이 틀렸다"로 끝내지 않고 어떤 state owner와 boundary에서 의미가 갈라졌는지 기록한다.

OBSERVED 모든 call을 동일 retry하지 않고 user-waiting foreground 경로에 더 많은 복원력을 준다.

RECOMMENDATION attempt identity와 first-visible-event 시점을 event stream에 명시한다.

ALGORITHM A12 · effect와 운영

정상 경로 뒤에 남는 effect와 실패 뒤에 남는 상태를 구분한다

retryable error에서 delay와 context를 갱신해 operation을 다시 호출한다.

effect가 시작되고 관찰되는 경계

첫 token이나 tool_use가 consumer에 보였다면 새 attempt는 화면·journal의 중복 가능성을 가진다.

현재 설계가 선택한 trade-off와 비용

모든 call을 동일 retry하지 않고 user-waiting foreground 경로에 더 많은 복원력을 준다. 이 선택의 계산·운영 비용은 최대 R회 operation, 대기 시간은 backoff 합.로 기술되지만, 비용 표기만으로 중복 effect나 부분 상태의 의미가 해결되지는 않는다.

실패·중단 뒤에 실제로 남을 수 있는 상태

retry storm, partial output duplication, cache/model switch는 최종 성공만 보면 숨는다. 이때 함께 대입할 반례는 partial output 중복, tool_use 뒤 provider retry, retry storm, 그리고 cache model switch이다. 이들을 모두 하나의 "실패"로 접으면 retry와 cleanup이 무엇을 이어받아야 하는지 알 수 없다.

관측이 다음 판단에 답하게 하는 법

attempt id와 first-visible-event 시점, provider request id를 연결해야 한다. 로그의 존재보다 중요한 것은 이 관측값이 같은 요청·state·effect를 가리키며, 복구 또는 중단 결정을 실제로 바꿀 수 있는가이다.

ALGORITHM A12 · 현재 상태 장부

고정 snapshot에서 확인된 것과 아직 말할 수 없는 것을 함께 남긴다

이 장부는 앞의 내용을 압축한 요약이 아니라, 다음 독자가 현재 시스템을 다시 구성하기 위한 출발 상태다.

OBSERVED	attempt loop, backoff context, AbortSignal 고려가 구현되어 있다.
CONDITIONAL	오류 분류와 operation idempotency가 맞는 요청에서만 retry가 의미를 보존한다.
UNKNOWN	provider별 commit 의미와 production retry distribution은 snapshot으로 알 수 없다.
NEXT READ	새 retry 대상 추가 전에 partial visibility와 external effect 여부를 분류해야 한다.

다른 입력과 변경안에 전이해 볼 질문

1. 첫 text delta 뒤 연결이 끊기면 새 attempt의 text를 기존 partial과 합칠 것인가 교체할 것인가?
2. model switch가 일어나면 cache와 request identity를 어떤 attempt boundary로 기록해야 하는가?

이 설명을 확인할 코드 src/services/api/withRetry.ts:170-220 src/services/api/withRetry.ts:430-516

ALGORITHM A13 · 독자의 질문

Compaction as lossy migration 은 어떤 시스템 문제를 닫는가

context window를 줄이려면 과거를 버려야 하지만, 최근 파일·tool identity·plan·running task처럼 다음 행동을 결정하는 상태는 자연어 요약만으로 안전하게 복원되지 않는다.

코드가 필요해지는 이유

context window가 커졌을 때 과거를 줄이면서 tool identity·plan·recent files·running task를 보존한다.

OBSERVED — compact service가 summary, boundary marker, 보존 message, post-compact attachment를 조립한다.

INFERENCE — compaction은 표시 최적화가 아니라 canonical history에서 다음 query context로 가는 lossy migration이다.

이 절에서 답할 결정 질문

무엇을 summary로 압축하고 무엇을 구조적 marker와 attachment로 보존해야 다음 turn의 world model이 이어지는가?

이 설명을 확인할 코드 src/services/compact/compact.ts:596-742 src/services/compact/compact.ts:596-628

ALGORITHM A13 · 선행 지식

문법 이름보다 먼저, 이 코드가 기대하는 실행 모델을 만든다

선행 지식은 용어 사전이 아니다. 아래 설명은 작은 실행 모델을 만든 다음, 그 모델이 src/services/compact/compact.ts의 결정점에서 어떤 오독을 막는지 되돌려 확인한다.

01
손실 압축과 checkpoint

모든 과거를 보존하지 않는 대신 재개에 필요한 최소 상태를 checkpoint로 남긴다. 자연어 summary는 의미를 설명하지만 exact identity와 version을 안정적으로 담기 어렵다.

> **실제 코드로 돌아가면.** 596–742행에서 boundary·summary·kept messages가 서로 어떤 정보를 맡는지 나눠 본다.

이론은 정답이 아니라 코드에 던질 질문이다

내구성·복구
`lossy state migration · checkpoint`
중단 지점에서 재개했을 때 무엇을 재생하고 무엇을 다시 실행하지 않아야 하는가?

구성·변형·성능
`semantic compression`
조립 방식이나 최적화를 바꿔도 핵심 계약이 adapter와 build 변형 안에서 보존되는가?

ALGORITHM A13 · 최소 예제

설명과 실행 전제를 같은 요약 문자열에 넣을 때 생기는 손실

파일 읽기와 작업 상태를 한 문장으로 줄인 뒤 원본이 바뀐 상황을 비교한다.

설명을 위한 최소 실행 예제

```
before = { file: 'a.ts', version: 'v3', task: 'running:t7' }
summary = 'a.ts를 읽었고 작업을 진행 중이다'
// 다음 turn에는 version과 task identity가 사라짐
restore = { file: 'a.ts', version: '?', task: '?' }
```

관찰 시점	값과 상태	프로젝트 코드에서의 의미
압축 전	file version과 task id가 구조화됨	stale 여부와 동일 작업을 판별할 수 있다.
자연어 요약	의미의 윤곽만 남음	정확한 identity를 다시 계산해야 한다.
재개	파일이 바뀌었을 수 있음	재관찰 attachment 또는 typed record가 필요하다.

예제를 실제 구현에 다시 연결한다

실제 구현의 boundary marker와 post-compact attachment가 summary 밖에서 어떤 실행 전제를 회복하는지 확인한다.

ALGORITHM A13 · 실제 코드

OBSERVED

Compaction as lossy migration 의 결정점은 어디에 있는가

요약은 compression이 아니라 future action을 위한 상태 재구성이다.

입력 표현. message history, custom instructions, ToolUseContext, 그리고 recompaction info

보존되는 작업 상태. summary, boundary marker, messages to keep, 그리고 post-compact attachments

boundary·summary message·post-compact 결과 구성

src/services/compact/compact.ts:596-742

```
596       // Create the compact boundary marker and summary messages before the
597       // event so we can compute the true resulting-context size.
598       const boundaryMarker = createCompactBoundaryMessage(
599         isAutoCompact ? 'auto' : 'manual',
600         preCompactTokenCount ?? 0,
601         messages.at(-1)?.uuid,
602       )
603       // Carry loaded-tool state — the summary doesn't preserve tool_reference
604       // blocks, so the post-compact schema filter needs this to keep sending
605       // already-loaded deferred tool schemas to the API.
606       const preCompactDiscovered = extractDiscoveredToolNames(messages)
607       if (preCompactDiscovered.size > 0) {
608         boundaryMarker.compactMetadata.preCompactDiscoveredTools = [
609           ...preCompactDiscovered,
… 130 lines omitted; use the source link for the complete range …
740         summaryMessages,
741         attachments: postCompactFileAttachments,
742         hookResults: hookMessages,
```

이 발췌는 고정 commit의 실제 코드다. 다음 페이지에서는 문장 순서를 되풀이하지 않고, 각 branch가 상태의 의미를 어떻게 좁히는지 실행 전·중·후로 다시 읽는다.

ALGORITHM A13 · 상태 전이

호출 순서가 아니라, 판단 근거가 어떻게 변하는지를 추적한다

message history, custom instructions, ToolUseContext, 그리고 recompaction info은 처음부터 신뢰된 실행 상태가 아니다. summary, boundary marker, messages to keep, 그리고 post-compact attachments가 앞선 관찰을 보존하기 때문에 뒤의 branch가 같은 작업을 계속 판단할 수 있다.

전제 정렬	**진입 조건을 해석한다** pre-compact hook와 status를 시작한다. 이 결과가 다음 판단 "요약용 context와 prompt를 만든다."의 전제가 된다.
표현 정규화	**표현과 선택지를 좁힌다** 요약용 context와 prompt를 만든다. 이 결과가 다음 판단 "prompt-too-long이면 제한된 truncation retry를 한다."의 전제가 된다.
유효성 판정	**표현과 선택지를 좁힌다** prompt-too-long이면 제한된 truncation retry를 한다. 이 결과가 다음 판단 "summary와 compact boundary를 만든다."의 전제가 된다.
결정 확정	**표현과 선택지를 좁힌다** summary와 compact boundary를 만든다. 이 결과가 다음 판단 "plan·skills·files·async agent attachment를 재주입한다."의 전제가 된다.
효과 조정	**표현과 선택지를 좁힌다** plan·skills·files·async agent attachment를 재주입한다. 이 결과가 다음 판단 "usage와 transcript reduction을 기록한다."의 전제가 된다.
상태 인계	**후속 owner가 읽을 상태를 남긴다** usage와 transcript reduction을 기록한다. 이 결과를 다음 owner가 읽을 수 있을 때 이 절차가 닫힌다.

이 전이가 끝난 뒤 기대할 수 있는 조건

tool_use identity와 unresolved work를 자연어 요약만으로 대체하지 않는다. boundary 이후 projection이 resume에서도 재구성 가능해야 한다. 중단된 compaction을 성공으로 기록하지 않는다.

ALGORITHM A13 · 설명의 보장 경계

이 구현이 지키려는 조건과, 설명이 멈춰야 하는 조건

여기서 불변조건은 형식적 증명의 결론이 아니다. 어느 전이가 의도한 의미를 보존했는지 읽고, 어떤 반례를 넣어 설명의 범위를 확인할지를 정하는 학습 도구다.

전이 전후에 확인할 조건

- **내구성·복구의 질문으로 읽기.** tool_use identity와 unresolved work를 자연어 요약만으로 대체하지 않는다.
- **구성·변형·성능의 질문으로 읽기.** boundary 이후 projection이 resume에서도 재구성 가능해야 한다.
- **내구성·복구의 질문으로 읽기.** 중단된 compaction을 성공으로 기록하지 않는다.

최종 출력만 같아도 중간 identity나 effect가 달라질 수 있으므로, 각 조건을 보존하는 transition을 함께 관찰해야 한다.

설명의 범위를 드러내는 반례

- 결정 이유 손실
- 오래된 file 내용 재주입
- 반복 compaction의 정보 손실 누적

이 반례가 나타나면 “알고리즘이 틀렸다”로 끝내지 않고 어떤 state owner와 boundary에서 의미가 갈라졌는지 기록한다.

OBSERVED 모든 과거를 유지할 수 없으므로 구조적 marker와 자연어 summary를 결합한다.

RECOMMENDATION action-critical state를 typed compact record로 분리하고 summary는 설명 layer로 제한한다.

ALGORITHM A13 · effect와 운영

정상 경로 뒤에 남는 effect와 실패 뒤에 남는 상태를 구분한다

history boundary를 고르고 summary를 생성한 뒤 보존 message와 attachment로 새 context를 만든다.

effect가 시작되고 관찰되는 경계

canonical transcript는 남아도 다음 model request가 보는 world model은 달라진다.

현재 설계가 선택한 trade-off와 비용

모든 과거를 유지할 수 없으므로 구조적 marker와 자연어 summary를 결합한다. 이 선택의 계산·운영 비용은 history 크기에 선형인 preparation + 별도 model call.로 기술되지만, 비용 표기만으로 중복 effect나 부분 상태의 의미가 해결되지는 않는다.

실패·중단 뒤에 실제로 남을 수 있는 상태

반복 compaction, stale file 재주입, 결정 이유 누락은 실행 가능한 잘못된 기억을 만든다. 이때 함께 대입할 반례는 결정 이유 손실, 오래된 file 내용 재주입, 그리고 반복 compaction의 정보 손실 누적이다. 이들을 모두 하나의 "실패"로 접으면 retry와 cleanup이 무엇을 이어받아야 하는지 알 수 없다.

관측이 다음 판단에 답하게 하는 법

compaction generation, source range, kept ids, restored resource versions를 남겨야 손실을 비교할 수 있다. 로그의 존재보다 중요한 것은 이 관측값이 같은 요청·state·effect를 가리키며, 복구 또는 중단 결정을 실제로 바꿀 수 있는가이다.

ALGORITHM A13 · 현재 상태 장부

고정 snapshot에서 확인된 것과 아직 말할 수 없는 것을 함께 남긴다

이 장부는 앞의 내용을 압축한 요약이 아니라, 다음 독자가 현재 시스템을 다시 구성하기 위한 출발 상태다.

OBSERVED	summary와 구조적 boundary, 보존 message, post-compact attachment가 결합된다.
CONDITIONAL	선택한 keep/restore 규칙이 현재 task에 필요한 identity를 포함할 때 재개 의미가 유지된다.
UNKNOWN	자연어 summary가 실제 model 행동에 미치는 정확한 손실과 반복 압축 품질은 확정할 수 없다.
NEXT READ	action-critical state를 typed compact record로 분리하기 전 compaction generation별 누락을 replay test로 수집해야 한다.

다른 입력과 변경안에 전이해 볼 질문

1. 두 번째 compaction이 첫 summary를 다시 요약하면 어떤 identity가 더 이상 source transcript로 연결되지 않는가?
2. 최근 파일이 삭제됐다면 attachment restore는 성공·skip·error 중 어떤 disposition을 남겨야 하는가?

이 설명을 확인할 코드 src/services/compact/compact.ts:596-742 src/services/compact/compact.ts:596-628

SOURCE READING · 1/2

OBSERVED

Budget check 위치가 policy 의미를 만든다

같은 숫자 제한도 model call 전후와 tool effect 전후 중 어디서 검사하느냐에 따라 결과가 달라진다.

token budget support src/query/tokenBudget.ts:1-60

```
1     import { getBudgetContinuationMessage } from '../utils/tokenBudget.js'
2
3     const COMPLETION_THRESHOLD = 0.9
4     const DIMINISHING_THRESHOLD = 500
5
6     export type BudgetTracker = {
7       continuationCount: number
8       lastDeltaTokens: number
9       lastGlobalTurnTokens: number
10      startedAt: number
11    }
12
13    export function createBudgetTracker(): BudgetTracker {
14      return {
15        continuationCount: 0,
16        lastDeltaTokens: 0,
17        lastGlobalTurnTokens: 0,
18        startedAt: Date.now(),
19      }
20    }
21
22    type ContinueDecision = {
23      action: 'continue'
24      nudgeMessage: string
25      continuationCount: number
… 30 lines omitted; use the source link for the complete range …
56      const pct = Math.round((turnTokens / budget) * 100)
57      const deltaSinceLastCheck = globalTurnTokens - tracker.lastGlobalTurnTokens
58
59      const isDiminishing =
60        tracker.continuationCount >= 3 &&
```

> 이 발췌는 설명을 위해 재작성한 의사코드가 아니라 고정 commit에서 기계적으로 가져온 실제 줄이다. 긴 범위는 가운데만 생략했으며 링크에서 전체 범위를 확인할 수 있다.

SOURCE READING · 2/2

OBSERVED

문법 아래에 숨은 상태와 책임

앞의 코드는 짧은 문법 표현을 사용하지만, 그 표현이 보장하는 범위와 프로젝트가 기대하는 범위는 같지 않다.

줄을 따라 확인할 관찰점

먼저 볼 표현	측정 값과 추정 값이 다를 수 있다.
그 표현이 만든 조건	compaction 전후 history visibility가 달라진다.
그 표현이 만든 조건	이미 시작된 tool effect는 budget으로 롤백되지 않는다.

언어와 런타임은 여기까지 보장한다

budget은 accounting과 control flow를 연결한다.

프로젝트가 이 메커니즘에 맡긴 책임

보고서는 threshold보다 check point와 terminal representation을 설명해야 한다.

경계를 잘못 읽을 때 생기는 위험

초과를 늦게 감지하면 새 effect가 시작될 수 있다.

이 설명을 확인할 코드 src/query/tokenBudget.ts:1-60

DISTINCTION TABLE

OBSERVED

Retry 가능성 판정

외부 관찰 commit point 이후에는 같은 시도를 반복한다고 볼 수 없다.

실패 시점	관찰된 것	권장 처리
request 전	없음	동일 attempt retry 가능
first delta 전	비용 가능	request id와 billing 확인
partial text 후	consumer가 일부 관찰	새 attempt 표지
tool_use 후	effect 가능	자동 replay 금지/조정

이 표를 읽는 관점

idempotency는 HTTP method가 아니라 시스템 전체의 관찰과 effect를 기준으로 판단한다.

ALGORITHM A14 · 독자의 질문

Message normalization lookup 구축은 어떤 시스템 문제를 닫는가

긴 history에서 매번 tool sibling과 progress 관계를 선형 탐색하면 consumer 수만큼 비용과 해석 차이가 늘어난다. 한 번 만든 lookup은 빠르지만 원본 snapshot과 함께 묶이지 않으면 stale하다.

코드가 필요해지는 이유

긴 history에서 tool sibling, progress, unresolved hook 관계를 반복 scan하지 않는다.

OBSERVED — message normalization이 tool-use/result/sibling/progress map을 한 snapshot에서 구축한다.

INFERENCE — lookup은 canonical message가 아니라 그 배열에서 파생된 read model이다.

이 절에서 답할 결정 질문

관계 index는 어떤 snapshot을 전제로 하며 mutable array가 바뀐 뒤 언제 폐기돼야 하는가?

> ### 이 코드를 읽기 전에 잡을 개념
>
> **index와 snapshot consistency.** index는 원본의 특정 시점에 대한 빠른 질의 구조다. 원본 identity나 version 없이 재사용하면 빠르게 틀린 답을 낸다.
>
> 1170–1215행에서 한 pass로 채우는 map과 duplicate id 처리 여부를 확인한다.

이 설명을 확인할 코드 src/utils/messages.ts:1170-1215

ALGORITHM A14 · 실제 코드

OBSERVED

Message normalization lookup 구축의 결정점은 어디에 있는가

반복적인 sibling/tool 조회를 index로 바꿔 대화 history를 선형에 가깝게 처리한다.

입력 표현. NormalizedMessage[]

보존되는 작업 상태. tool-use map, result map, sibling groups, 그리고 progress map

message lookup construction

src/utils/messages.ts:1170-1215

```
1170  export function buildMessageLookups(
1171    normalizedMessages: NormalizedMessage[],
1172    messages: Message[],
1173  ): MessageLookups {
1174    // First pass: group assistant messages by ID and collect all tool use IDs per message
1175    const toolUseIDsByMessageID = new Map<string, Set<string>>()
1176    const toolUseIDToMessageID = new Map<string, string>()
1177    const toolUseByToolUseID = new Map<string, ToolUseBlockParam>()
1178    for (const msg of messages) {
1179      if (msg.type === 'assistant') {
1180        const id = msg.message.id
1181        let toolUseIDs = toolUseIDsByMessageID.get(id)
1182        if (!toolUseIDs) {
1183          toolUseIDs = new Set()
… 29 lines omitted; use the source link for the complete range …
1213
1214    for (const msg of normalizedMessages) {
1215      if (msg.type === 'progress') {
```

이 발췌는 고정 commit의 실제 코드다. 다음 페이지에서는 문장 순서를 되풀이하지 않고, 각 branch가 상태의 의미를 어떻게 좁히는지 실행 전·중·후로 다시 읽는다.

ALGORITHM A14 · 상태 전이

호출 순서가 아니라, 판단 근거가 어떻게 변하는지를 추적한다

NormalizedMessage[]은 처음부터 신뢰된 실행 상태가 아니다. tool-use map, result map, sibling groups, 그리고 progress map가 앞선 관찰을 보존하기 때문에 뒤의 branch가 같은 작업을 계속 판단할 수 있다.

전제 정렬	진입 조건을 해석한다	한 번 순회하며 type/id별 index를 만든다. 이 결과가 다음 판단 "후속 projection이 lookup을 공유한다."의 전제가 된다.
표현 정규화	표현과 선택지를 좁힌다	후속 projection이 lookup을 공유한다. 이 결과가 다음 판단 "새 message history에는 새 lookup을 만든다."의 전제가 된다.
상태 인계	후속 owner가 읽을 상태를 남긴다	새 message history에는 새 lookup을 만든다. 이 결과를 다음 owner가 읽을 수 있을 때 이 절차가 닫힌다.

이 전이가 끝난 뒤 기대할 수 있는 조건

index는 정확히 같은 immutable message snapshot에 대응한다. UUID/tool id collision을 조용히 덮지 않는다.

ALGORITHM A14 · 설명의 보장 경계

이 구현이 지키려는 조건과, 설명이 멈춰야 하는 조건

여기서 불변조건은 형식적 증명의 결론이 아니다. 어느 전이가 의도한 의미를 보존했는지 읽고, 어떤 반례를 넣어 설명의 범위를 확인할지를 정하는 학습 도구다.

전이 전후에 확인할 조건

- **구성·변형·성능의 질문으로 읽기.** index는 정확히 같은 immutable message snapshot에 대응한다.
- **동시성·시간·합의의 질문으로 읽기.** UUID/tool id collision을 조용히 덮지 않는다.

최종 출력만 같아도 중간 identity나 effect가 달라질 수 있으므로, 각 조건을 보존하는 transition을 함께 관찰해야 한다.

설명의 범위를 드러내는 반례

- mutable array 변경 뒤 stale lookup
- 같은 id 중복
- 새 variant가 index에서 누락

이 반례가 나타나면 “알고리즘이 틀렸다”로 끝내지 않고 어떤 state owner와 boundary에서 의미가 갈라졌는지 기록한다.

OBSERVED 여러 UI/API consumer가 같은 관계 계산을 재사용한다.

RECOMMENDATION lookup 생성과 message snapshot identity를 함께 캐시하고 mutation을 제한한다.

ALGORITHM A14 · 운영과 현재 상태

작은 변환도 effect와 미확정 경계를 남긴다

message 배열을 순회해 여러 관계 map을 만든 뒤 UI/API consumer가 공유한다.

실패 뒤 남는 상태

mutable 배열 변경, duplicate id, 새 variant 누락은 lookup과 source가 갈라지게 한다.

OBSERVED	여러 relation map을 단일 construction pass에서 만든다.
CONDITIONAL	message 배열이 이후 변하지 않거나 변경 시 lookup을 재생성할 때 정확하다.
UNKNOWN	모든 caller가 immutable snapshot discipline을 지키는지는 확인되지 않는다.
NEXT READ	memoization 도입 전 배열 identity와 mutation site를 먼저 찾아야 한다.

다시 적용할 질문

1. 같은 tool id가 두 번 나오면 map의 마지막 write가 숨기는 것은 무엇인가?
2. message 한 개를 in-place mutate하면 lookup cache key가 그 변화를 관찰할 수 있는가?

SELECTIVE EXECUTION TRACE

OBSERVED

3권에서 보존한 사례 상태

4권으로 넘어갈 때 필요한 정보만 명시한다.

책임 연결

Message

원래 user request와 source UUID

이때 남은 상태와 판단이 "Read"에서 사용할 전제가 된다.

책임 연결

Read

r1 결과와 file state/version

이때 남은 상태와 판단이 "Edit proposal"에서 사용할 전제가 된다.

책임 연결

Edit proposal

e1 id, path, old/new string

이때 남은 상태와 판단이 "Authority"에서 사용할 전제가 된다.

책임 연결

Authority

현재 permission context

이때 남은 상태와 판단이 "Control"에서 사용할 전제가 된다.

책임 연결

Control

AbortSignal과 budget 상태

이 지점에서 앞선 상태 변화가 외부에서 관찰 가능한 결과로 정산된다.

이 흐름 전체에서 보존해야 하는 것

4권의 effect pipeline은 이 identity를 유지한 채 최종 executable input을 결정해야 한다.

어디에서 설명이 틀어지는가

승인 후 input이 다시 바뀌면 사용자가 본 preview와 effect가 달라진다.

이 설명을 확인할 코드 src/Tool.ts:158-205

THEORY ATLAS · 1/2

용어를 외우지 않고, 어떤 보장을 설명하는 이론인지 묶어 읽는다

이 지도는 이 권의 알고리즘이 실제로 참조한 이론만 집계한다. 빈도는 중요도 점수가 아니라 여러 구현 결정을 같은 질문으로 비교할 수 있는 탐색 신호다.

상태·계산 모델

4개 알고리즘 · 5개 표현

입력이 어떤 중간 표현과 상태 전이를 거쳐 결과가 되는지, 그리고 각 전이 전후에 무엇이 계속 참이어야 하는지를 설명한다.

이 권에서 나타나는 방식. A07 사용자 입력 dispatch과 A09 API message normalization에서 이 관점이 결정적인 이유는 “local-only output을 provider input과 구분한다. 또한 각 tool_result는 대응 tool_use 뒤에 온다.”라는 조건을 구현 세부사항보다 오래 유지해야 하기 때문이다.

참조 분포. interpreter(A07), command pattern(A07), normal form(A09), state machine(A10·A39), transition invariant(A10)

스터디 질문. 단계의 순서나 표현을 바꿔도 같은 상태와 불변조건에 도달하는가?

동시성·시간·합의

2개 알고리즘 · 3개 표현

여러 실행이 같은 상태를 읽고 쓰는 동안 순서, 취소, 재시도, 관찰 시점이 결과의 의미를 바꾸지 않게 하는 조건을 다룬다.

이 권에서 나타나는 방식. A12 API retry와 backoff과 A14 Message normalization lookup 구축에서 이 관점이 결정적인 이유는 “abort는 새 retry를 시작하지 않는다. 또한 index는 정확히 같은 immutable message snapshot에 대응한다.”라는 조건을 구현 세부사항보다 오래 유지해야 하기 때문이다.

참조 분포. exponential backoff(A12), retry budget(A12), snapshot consistency(A14)

스터디 질문. stale read, 중복 실행, 취소가 끼어들어도 effect와 최종 상태의 의미가 보존되는가?

프로토콜·식별

5개 알고리즘 · 6개 표현

서로 다른 계층이 같은 작업과 같은 메시지를 가리킨다는 사실을 타입, 식별자, completion 규약으로 유지한다.

이 권에서 나타나는 방식. A07 사용자 입력 dispatch과 A08 System/User/System context 조립에서 이 관점이 결정적인 이유는 “local-only output을 provider input과 구분한다. 또한 순서가 바뀌면 instruction scope와 cache prefix가 달라질 수 있다.”라는 조건을 구현 세부사항보다 오래 유지해야 하기 때문이다.

참조 분포. tagged union(A07·A39), compiler lowering(A08), lossy projection(A09), correlation id(A11), protocol completion(A11), referential integrity(A39)

스터디 질문. 생산자와 소비자가 동일한 값·작업·종료 의미를 해석한다는 증거가 있는가?

괄호 안의 알고리즘 ID로 돌아가면 이론의 정의가 아니라 그 이론이 설명하는 source transition, 반례, 불변조건을 함께 확인할 수 있다.

THEORY ATLAS · 2/2

용어를 외우지 않고, 어떤 보장을 설명하는 이론인지 묶어 읽는다

이 지도는 이 권의 알고리즘이 실제로 참조한 이론만 집계한다. 빈도는 중요도 점수가 아니라 여러 구현 결정을 같은 질문으로 비교할 수 있는 탐색 신호다.

내구성·복구

3개 알고리즘 · 4개 표현

부분 기록, 재시작, 버전 변화가 있어도 이전 사실을 잃거나 같은 effect를 다시 확정하지 않도록 만드는 복구 계약을 설명한다.

이 권에서 나타나는 방식. A10 queryLoop 상태 전이과 A12 API retry와 backoff에서 이 관점이 결정적인 이유는 “tool_use/tool_result pairing 없이 다음 provider call로 가지 않는다. 또한 abort는 새 retry를 시작하지 않는다.”라는 조건을 구현 세부사항보다 오래 유지해야 하기 때문이다.

참조 분포. event sourcing(A10), idempotency/commit point(A12), lossy state migration(A13), checkpoint(A13)

스터디 질문. 중단 지점에서 재개했을 때 무엇을 재생하고 무엇을 다시 실행하지 않아야 하는가?

구성·변형·성능

4개 알고리즘 · 6개 표현

동일한 핵심 의미를 여러 실행 표면과 제품 변형에 조립하면서 비용과 의존성의 위치를 통제하는 방법을 다룬다.

이 권에서 나타나는 방식. A08 System/User/System context 조립과 A09 API message normalization에서 이 관점이 결정적인 이유는 “순서가 바뀌면 instruction scope와 cache prefix가 달라질 수 있다. 또한 각 tool_result는 대응 tool_use 뒤에 온다.”라는 조건을 구현 세부사항보다 오래 유지해야 하기 때문이다.

참조 분포. configuration composition(A08), prompt caching(A08), anti-corruption layer(A09), semantic compression(A13), indexing(A14), memoization(A14)

스터디 질문. 조립 방식이나 최적화를 바꿔도 핵심 계약이 adapter와 build 변형 안에서 보존되는가?

검증·관측·운영

1개 알고리즘 · 1개 표현

실행 결과를 관찰하고 회귀를 검출하며, cleanup과 운영 신호가 본래 동작을 왜곡하지 않도록 만드는 증거 체계를 다룬다.

이 권에서 나타나는 방식. A11 tool-use/tool-result pairing에서 이 관점이 결정적인 이유는 “unknown tool도 silent drop하지 않는다.”라는 조건을 구현 세부사항보다 오래 유지해야 하기 때문이다.

참조 분포. saga observation(A11)

스터디 질문. 최종 출력뿐 아니라 실패·중단·복구 경로의 보장을 어떤 관찰과 테스트로 입증할 수 있는가?

괄호 안의 알고리즘 ID로 돌아가면 이론의 정의가 아니라 그 이론이 설명하는 source transition, 반례, 불변조건을 함께 확인할 수 있다.

ALGORITHM INDEX

OBSERVED

이 권의 결정적 알고리즘과 보장

알고리즘 이름보다 마지막 열을 먼저 읽는다. 프로젝트에서 중요한 것은 계산 기법 자체보다 그 계산으로 어떤 시스템 조건을 계속 참으로 만드는가이기 때문이다.

ID	문제와 구현	대표 불변조건
A07	**사용자 입력 dispatch** 하나의 입력 표면에서 local effect와 model-bound message를 안전하게 구분한다.	local-only output을 provider input과 구분한다.
A08	**System/User/System context 조립** 도구, 모델, cwd, memory, policy에서 온 조각을 provider 요청에 맞는 context로 만든다.	순서가 바뀌면 instruction scope와 cache prefix가 달라질 수 있다.
A09	**API message normalization** UI-only/system metadata와 불완전한 tool state를 제거·병합해 API protocol을 만족시킨다.	각 tool_result는 대응 tool_use 뒤에 온다.
A10	**queryLoop 상태 전이** 여러 model request와 tool phase를 한 user turn 안에서 반복하면서 recovery·compact·budget 상태를 보존한다.	tool_use/tool_result pairing 없이 다음 provider call로 가지 않는다.
A11	**tool-use/tool-result pairing** model이 요청한 tool call을 실행 가능한 tool과 연결하고 성공·거부·오류를 동일 protocol로 반환한다.	unknown tool도 silent drop하지 않는다.
A39	**부분 content block의 typed reconstruction** text, thinking, tool input JSON, connector block이 서로 다른 delta로 도착하는 동안 type과 index를 잃지 않고 내부 message를 만든다.	delta index에는 먼저 시작된 block이 존재한다.
A12	**API retry와 backoff** 일시적 provider 실패에서 cache·latency·rate limit을 고려해 요청을 재시도한다.	abort는 새 retry를 시작하지 않는다.
A13	**Compaction as lossy migration** context window가 커졌을 때 과거를 줄이면서 tool identity·plan·recent files·running task를 보존한다.	tool_use identity와 unresolved work를 자연어 요약만으로 대체하지 않는다.
A14	**Message normalization lookup 구축** 긴 history에서 tool sibling, progress, unresolved hook 관계를 반복 scan하지 않는다.	index는 정확히 같은 immutable message snapshot에 대응한다.

SOURCE EVIDENCE INDEX · 1/2

OBSERVED

설명을 다시 코드로 되돌리는 출발점

보고서의 해석은 아래 고정 줄 범위에서 다시 검토할 수 있다. 링크는 모두 동일한 commit을 가리키며, 파일 이름만 비슷한 최신 branch로 이동하지 않는다.

고정 코드 범위	연결되는 설명	표현 방식
src/utils/processUserInput/processUserInput.ts:515-588	사용자 입력 dispatch	algorithm
src/utils/processUserInput/processUserInput.ts:281-330	사용자 입력 dispatch	algorithm
src/utils/queryContext.ts:44-87	System/User/System context 조립	algorithm
src/entrypoints/sdk/coreSchemas.ts:1854-1882	Discriminated union과 runtime schema는 서로 다른 보증을 맡는다	code
src/utils/messages.ts:1989-2028	API message normalization	algorithm
src/utils/claudemd.ts:1-26	CLAUDE.md discovery는 directory walk와 source priority를 결합한다	code
src/utils/claudemd.ts:820-936	CLAUDE.md discovery는 directory walk와 source priority를 결합한다	code
src/outputStyles/loadOutputStylesDir.ts:1-79	Output style은 답변 형식만이 아니라 coding instruction 보존 여부를 바꾼다	code
src/skills/loadSkillsDir.ts:50-135	Skill frontmatter는 discovery metadata와 실행 capability를 함께 선언한다	code
src/skills/loadSkillsDir.ts:136-239	Skill frontmatter는 discovery metadata와 실행 capability를 함께 선언한다	code
src/query/tokenBudget.ts:1-80	Token budget은 quota 차감이 아니라 turn continuation policy다	code
src/services/tokenEstimation.ts:84-175	정확한 token count와 rough estimate는 실패 mode가 다르다	code
src/cost-tracker.ts:55-143	모델 실행을 결정하는 수치를 분리한다	matrix
src/query.ts:219-239	yield는 여러 번 반환하는 문법이 아니다	mechanism-study
src/screens/REPL.tsx:2788-2804	yield는 여러 번 반환하는 문법이 아니다	mechanism-study
src/QueryEngine.ts:675-686	yield는 여러 번 반환하는 문법이 아니다	mechanism-study
src/query.ts:1260-1266	yield는 여러 번 반환하는 문법이 아니다	mechanism-study
src/query.ts:1508-1522	yield는 여러 번 반환하는 문법이 아니다	mechanism-study
src/query.ts:219-250	query()의 event와 return은 다른 채널이다	code
src/query.ts:252-321	queryLoop 상태 전이	algorithm
src/services/tools/toolExecution.ts:337-410	tool-use/tool-result pairing	algorithm
src/services/api/claude.ts:752-790	Streaming은 텍스트가 아니라 typed event transport다	code
src/tools/FileReadTool/FileReadTool.ts:337-380	사례의 첫 model/tool 왕복	walkthrough
src/tools/FileEditTool/FileEditTool.ts:86-130	사례의 첫 model/tool 왕복	walkthrough

SOURCE EVIDENCE INDEX · 2/2

OBSERVED

설명을 다시 코드로 되돌리는 출발점

보고서의 해석은 아래 고정 줄 범위에서 다시 검토할 수 있다. 링크는 모두 동일한 commit을 가리키며, 파일 이름만 비슷한 최신 branch로 이동하지 않는다.

고정 코드 범위	연결되는 설명	표현 방식
src/services/api/claude.ts:752-774	queryModelWithStreaming은 provider adapter를 감싸는 protocol 입구다	code
src/services/api/claude.ts:1940-2054	부분 content block의 typed reconstruction	algorithm
src/services/api/claude.ts:2053-2210	부분 content block의 typed reconstruction	algorithm
src/services/api/claude.ts:2211-2302	부분 content block의 typed reconstruction	algorithm
src/services/api/claude.ts:2211-2251	message_delta는 이미 yield된 message의 마지막 metadata를 정산한다	code
src/services/api/claude.ts:1880-1949	stream idle watchdog는 중단과 정상 완료를 구분한다	code
src/services/api/claude.ts:2053-2125	tool_use JSON이 실행 가능한 input이 되기까지	walkthrough
src/services/api/withRetry.ts:170-220	API retry와 backoff	algorithm
src/services/api/withRetry.ts:430-516	API retry와 backoff	algorithm
src/services/compact/compact.ts:596-742	Compaction as lossy migration	algorithm
src/services/compact/compact.ts:596-628	Compaction as lossy migration	algorithm
src/query/tokenBudget.ts:1-60	Budget check 위치가 policy 의미를 만든다	code
src/utils/messages.ts:1170-1215	Message normalization lookup 구축	algorithm
src/Tool.ts:158-205	3권에서 보존한 사례 상태	walkthrough

STUDY SYNTHESIS

Turn protocol 검증

읽은 내용을 요약하는 대신, 새로운 변경이나 장애 상황에서도 같은 보장을 찾아낼 수 있는지 확인한다.

검증 질문	yield event와 generator return을 구분한다.
검증 질문	provider content block의 start·delta·stop과 늦은 message_delta를 재구성한다.
검증 질문	tool_use/tool_result pairing 불변조건을 적는다.
검증 질문	retry commit point 네 단계를 사례에 적용한다.
검증 질문	compaction에서 typed state와 summary를 나눈다.

이 권의 완료 기준

한 turn을 단순 “모델→도구 반복”이 아니라 provider partial state, message identity, effect pairing, terminal settlement가 연결된 protocol로 설명한다.

답에는 반드시 현재 snapshot의 코드 근거, 그 근거에서 도출한 해석, 아직 확인할 수 없는 부분을 구분해 적는다.